



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPARTMENT OF DATA SCIENCE AND ENGINEERING

EngageMind: AI/ML-Based Language Model Improved on GPT-2, Incorporated in an Interactive, Authenticated App

Supervisor:

Bencsik Gergely

Assistant Professor

Author:

Haseeb Raza

Computer Science BSc

Budapest, 2025

Contents

1	Introduction	5
	Introduction	5
	Background and Motivation	6
	Problem Statement	7
	Aims and Objectives	8
	Scope and Methodology	9
2	User Documentation	11
2.1	Install Guide	11
2.1.1	System Requirements	11
2.1.2	Installation Steps	12
2.2	Configuration and Setup	14
2.2.1	Environment Variables	14
2.2.2	Database Setup	14
2.2.3	Starting Services	15
2.3	User Interaction with the EngageMind RAG Application	16
2.3.1	Introduction to the Frontend Interface	16
2.3.2	User Registration	16
2.3.3	Logging In	19
2.3.4	Chatting with the RAG System	21
2.3.5	Troubleshooting	25
2.3.6	Example Commands	27

3	Developer Documentation	29
4	Developer Documentation	30
4.1	Cloning the Repository	30
4.1.1	Prerequisites for Cloning	30
4.1.2	Repository Details	31
4.1.3	Cloning the Repository	31
4.1.4	Verifying the Local Repository	32
4.1.5	Repository Structure Overview	32
4.1.6	Handling Repository Updates	32
4.1.7	Managing Submodules (If Applicable)	32
4.1.8	Troubleshooting Common Issues	33
4.2	Prerequisites	33
4.2.1	MongoDB (Database)	33
4.2.2	Flask Backend (RAG + Inference Engine)	34
4.2.3	Express Backend (User Authentication)	35
4.2.4	React Frontend	36
4.2.5	Background Services: Celery and Redis	37
4.3	External APIs Used	38
4.3.1	Mistral API	38
4.3.2	Tavily Web Search API	39
4.3.3	Resend API (Email Delivery)	39
4.3.4	Google OAuth (Passport.js)	40
4.4	Software Architecture	41
4.4.1	System Overview	41
4.4.2	Architecture Diagram	42
4.4.3	System Logic Overview	42
4.4.4	RAG System Details	43

4.4.5	LangGraph Memory Workflow (<code>app.py</code>)	44
4.4.6	Authentication System	45
4.4.7	Database Design (MongoDB)	45
4.4.8	API Surface	46
4.4.9	Real-Time & Evaluation	46
4.4.10	Deployment Considerations	47
4.5	RAG Implementation	47
4.5.1	Overview of RAG backend	47
4.5.2	Ingestion Pipeline	47
4.5.3	Retrieval Pipeline	51
4.5.4	Server and API Setup	56
4.5.5	FAISS Index Management	60
4.5.6	Evaluation Module	62
4.5.7	Summary of RAG Backend	64
4.6	UserAuth Backend Implementation	66
4.6.1	User Authentication Backend Summary	70
4.7	Frontend Implementation	71
4.7.1	Project Overview	71
4.7.2	Application Bootstrapping: <code>index.js</code>	71
4.7.3	Routing Architecture: <code>App.js</code>	72
4.7.4	User Authentication Flow	72
4.7.5	ProtectedRoute Logic	75
4.7.6	Chat System Detailed Flow	76
4.7.7	Components Connection	79
4.7.8	Loading and Error Handling	79
4.7.9	LocalStorage Key List	80
4.7.10	Data Flow Diagram	80
4.7.11	Chat Application Flow	80

Acknowledgements	82
Bibliography	82
List of Figures	85
List of Tables	86
List of Algorithms	87
List of Codes	88

Chapter 1

Introduction

Introduction

The rapidly evolving field of conversational AI holds immense potential for transforming how we interact with technology. Large Language Models (LLMs), such as GPT-2 [1], have demonstrated remarkable capabilities in generating text closely resembling human language, positioning them as powerful tools for AI assistants and interactive applications. Despite their impressive advancements, current models face significant limitations that hinder truly personalized and adaptive user experiences. A critical shortcoming is their inability to retain memory from past interactions, resulting in difficulties maintaining coherent and sustained dialogues. Additionally, most existing models cannot adapt dynamically to individual user preferences or to leverage user-specific data effectively, significantly reducing their potential in personalized scenarios. For instance, GPT-2's fixed context window restricts its memory to recent interactions, inevitably losing context as conversations extend beyond its set limitations.

To overcome these challenges, this thesis introduces **EngageMind**, an innovative conversational AI system built upon GPT-2, integrating enhanced functionalities for a more personalized and adaptive interaction. EngageMind incorporates robust user authentication to ensure secure, individualized access, coupled with a sophisticated memory retention system that stores and retrieves past conversations to maintain contextual continuity. Furthermore, EngageMind leverages **Retrieval-Augmented Generation (RAG)** techniques, allowing adaptive learning from user-uploaded documents to provide contextually relevant responses. Real-time monitoring tools are also integrated, offering

transparency and feedback during intensive operations such as model real-time response. By combining these advanced features, EngageMind endeavors to deliver smarter, more engaging interactions that evolve continuously alongside its users, moving closer to achieving truly meaningful and better communication with machines.

Background and Motivation

Conversational artificial intelligence has undergone transformative advancements with the development of **large language models (LLMs)** such as GPT-2, GPT-3, and GPT-4. These sophisticated models demonstrate remarkable proficiency in generating **coherent and contextually relevant text**, enabling a wide array of applications, from intuitive chatbots and virtual assistants to advanced research tools. Despite their impressive capabilities, a critical limitation persists: most LLMs are inherently **stateless**, lacking the ability to maintain **long-term memory** across multiple interactions. This constraint significantly hinders their capacity to dynamically adapt to individual user profiles and preferences, thereby limiting their effectiveness in **personalized, complex, and real-world scenarios**.

Modern digital ecosystems increasingly demand **highly personalized and context-aware interactions**. The inability of traditional LLMs to tailor responses, preserve conversation histories, or securely integrate user-specific documents poses a substantial barrier. For instance, effective educational platforms require AI systems capable of **continuously tracking and adapting** to a student’s learning trajectory. Similarly, enterprise applications necessitate **secure authentication** and the integration of proprietary data to deliver consistent, accurate, and domain-specific insights. While foundational models like GPT-2 excel in robust text generation, their architectures are not designed for **persistent memory storage, incremental learning, or real-time adaptive feedback loops**.

To address these challenges, we propose **EngageMind**, an innovative framework designed to augment foundational LLMs such as GPT-2. EngageMind incorporates es-

sential functionalities, including **secure user authentication**, **persistent memory storage** for retaining dialogue histories, and **adaptive training methodologies** driven by user-provided data. Through these enhancements, EngageMind seeks to transform conversational AI systems from generic, static models into **dynamic, personalized assistants**, capable of continuously refining and tailoring their responses to align with individual user needs and evolving contexts.

Problem Statement

The remarkable adaptability of modern **large language models (LLMs)** is constrained by significant limitations that impede the delivery of **truly personalized and contextually rich conversational experiences**. A primary challenge is the **absence of stateful memory**, as most conversational AI systems fail to retain dialogue histories across sessions, necessitating repetitive interactions. This stateless nature leads to **reduced user engagement, limited contextual understanding**, and interactions that feel disjointed and generic. Additionally, the **lack of personalization** in responses remains a critical issue, with current models rarely tailoring outputs to individual user profiles, past preferences, or explicit feedback, rendering them inadequate for applications requiring **user-specific, nuanced insights**.

Moreover, the practical deployment of conversational AI is hampered by **inadequate security and data management practices**. Many systems lack robust **user authentication** or secure mechanisms for handling sensitive data, raising significant concerns about **privacy, trust**, and applicability in confidential or domain-specific contexts. These challenges are compounded by **insufficient real-time monitoring and feedback mechanisms**, particularly during adaptive processes such as continuous training or model fine-tuning. Without **transparent, real-time oversight**, users and developers cannot effectively monitor or guide the evolving performance and alignment of AI systems.

To address these critical gaps, we propose **EngageMind**, a comprehensive framework

designed to integrate **secure user authentication**, **persistent knowledge storage**, and **dynamic personalization**. EngageMind enables **stateful memory** by securely retaining and retrieving interaction histories and user preferences, ensuring **contextually coherent conversations** across multiple sessions. Through **adaptive learning mechanisms**, it incorporates user-provided documents and feedback to continuously refine and specialize responses. By adhering to stringent **privacy and security standards**, EngageMind fosters user confidence and supports deployment in sensitive and specialized environments, overcoming the limitations of current conversational AI technologies.

Aims and Objectives

The primary goal of this thesis is to conceptualize, develop, and validate **EngageMind**, an advanced conversational AI framework that extends the capabilities of the GPT-2 model. EngageMind aims to address existing limitations by delivering **persistent conversational memory**, **dynamic personalization** driven by user-specific data, **secure interactions**, and **transparent operational monitoring**. To achieve this overarching objective, the following specific aims are defined:

- **Secure User Authentication and Session Management:** Develop a robust system to protect user interactions, ensuring personalized experiences within **secure, authenticated sessions**.
- **Persistent Contextual Memory:** Engineer reliable mechanisms for storing and retrieving dialogue histories, enabling **seamless continuity** in user interactions.
- **Adaptive Learning via Retrieval-Augmented Generation (RAG):** Integrate RAG methodologies to securely incorporate user-provided documents, facilitating **tailored, adaptive learning** aligned with individual needs.
- **Real-Time Operational Monitoring Interfaces:** Provide transparent visualization tools to monitor resource-intensive processes, such as training, enhancing **system manageability** and **user trust**.

- **Integrated Full-Stack System Architecture:** Construct a scalable, maintainable architecture leveraging **React** for the frontend, **Flask** or **Django** for backend services, **PyTorch** for the AI core, and **PostgreSQL** or **MongoDB** for efficient data management.
- **Rigorous Evaluation of Performance and User Satisfaction:** Conduct comprehensive quantitative and qualitative assessments to validate EngageMind’s ability to deliver **personalized, coherent, and contextually relevant responses**, ensuring practical usability through structured user feedback and testing.

Scope and Methodology

This research focuses on the design, development, and demonstration of **EngageMind** as a functional proof-of-concept system, enhancing conversational AI through **personalized memory retention** and **adaptive learning** built upon GPT-2. Core functionalities include **secure authentication**, robust memory modules for storing user-specific dialogue histories, **RAG integration** for contextually relevant responses based on user-uploaded documents, and **user feedback mechanisms** for continuous improvement. The initial prototype will focus on **text-based interactions in English** and utilize GPT-2 due to its suitability for academic research. While generalizable, extensive multilingual support, quantitative benchmarking against state-of-the-art models, and comprehensive user interface refinements are beyond the current scope, enabling a focused exploration of **memory and adaptive learning** concepts.

EngageMind adopts a **modular, full-stack implementation methodology**. **PyTorch** will be employed for fine-tuning GPT-2 and implementing the RAG pipeline for document retrieval. The responsive frontend interface, developed with **React**, will support user authentication, chat interactions, document uploads, and feedback collection. Backend operations, managing API interactions and tasks such as retraining, will utilize lightweight frameworks like **Flask** or **Django**. Data persistence, critical for memory functionalities, will leverage databases such as **PostgreSQL** or **MongoDB**, with retrieval of

historical interactions employing **embedding-based similarity searches**. Development will follow an **iterative approach**, implementing and validating individual features before full integration, adhering to standard software development practices.

The expected outcome is a functional prototype enabling authenticated users to engage in **personalized, context-aware interactions** with a chatbot capable of **retaining conversation histories, utilizing user documents**, and **continuously improving** via feedback. EngageMind aims to demonstrate the advantages of **long-term memory, document-driven context augmentation**, and **feedback-driven adaptation**, validating its approach and laying the foundation for scalable, advanced conversational AI solutions.

In conclusion, this chapter has laid the foundation for addressing the critical limitations of current conversational AI systems by introducing **EngageMind**, a novel framework designed to enhance GPT-2 with **persistent memory, dynamic personalization**, and **secure interactions**. By outlining the challenges of stateless LLMs, the need for context-aware and user-specific responses, and the proposed objectives of EngageMind, this chapter sets the stage for a comprehensive exploration of its design, implementation, and evaluation. The subsequent chapters will delve into the technical architecture, development methodology, and empirical validation of EngageMind, demonstrating its potential to redefine personalized conversational AI.

Chapter 2

User Documentation

2.1 Install Guide

This section provides a detailed guide for installing EngageMind, an AI/ML-based language model application. It covers system requirements and installation steps for all necessary components.

2.1.1 System Requirements

Before proceeding with the installation, ensure your system meets the following specifications:

Hardware Requirements

- **Processor:** Minimum 2.0 GHz dual-core; quad-core recommended
- **RAM:** At least 8 GB required; 16 GB or more recommended
- **Storage:** Minimum 20 GB of free disk space; SSD recommended
- **GPU (Optional):** CUDA-compatible GPU accelerates training and inference

Software Requirements

- **Operating System:** Linux (Ubuntu 20.04+), macOS (10.15+), or Windows 10/11
- **Python:** Version 3.8 or higher
- **Node.js:** Latest LTS version (16.x or 18.x)
- **Express:** For authentication and session management

- **Flask:** For the RAG backend and GPT-2 integration
- **React:** For the frontend interface
- **Git:** For repository management
- **Docker + Docker Compose (Optional):** For containerized deployment

2.1.2 Installation Steps

Installing Git

Install Git based on your operating system:

Windows:

1. Download the Git installer from <https://git-scm.com/download/win>
2. Run the installer with default settings
3. Verify with: `git -version`

macOS:

1. Open Terminal and type `git -version`
2. If not installed, macOS will prompt to install Xcode Command Line Tools

Linux:

```
1 sudo apt-get update
2 sudo apt-get install git -y
3 git --version
```

Cloning the Repository

Clone the EngageMind repository:

```
1 git clone https://github.com/haseebraza715/EngageMind.git
2 cd EngageMind
```

Setting Up Python and Flask

Set up the Flask backend:

```
1 # Verify Python installation
2 python --version
3
4 # Create and activate virtual environment
5 python -m venv venv
6 # For macOS/Linux:
7 source venv/bin/activate
8 # For Windows:
9 venv\Scripts\activate
10
11 # Install dependencies
12 pip install --upgrade pip
13 pip install -r requirements.txt
```

Setting Up Node.js and Express

Set up the Express authentication service:

```
1 # Verify Node.js installation
2 node --version
3 npm --version
4
5 # Install Express dependencies
6 cd auth_express
7 npm install
```

Setting Up React Frontend

Set up the React frontend:

```
1 # Install React dependencies
2 cd ../EngageMind-frontend
3 npm install
```

2.2 Configuration and Setup

After installation, configure environment variables, set up the database, and start services.

2.2.1 Environment Variables

Create environment files for both services:

Flask Backend Configuration (EngageMind_rag/.env):

```
1 MISTRAL_API_KEY=your_mistral_api_key
2 HF_TOKEN==your_huggingface_token
3 MISTRAL_API_ENDPOINT="https://api.mistral.com"
4 RAG_API_TOKEN=your_secret_token
```

Express Auth Configuration (EngageMind_backend/.env):

```
1 PORT=3001
2 SESSION_SECRET=your_express_session_secret
3 JWT_SECRET=your_jwt_secret
4 MONGO_URI=mongodb://localhost:27017/engagemindbackend
```

2.2.2 Database Setup

Set up MongoDB for user authentication:

```
1 # Install MongoDB
```

```
2 sudo apt-get install -y mongodb-org
3
4 # Start MongoDB service
5 sudo systemctl start mongod
6 sudo systemctl enable mongod
7
8 # Create database and user
9 mongosh
10 use engagemind_db
11 db.createUser({
12     user: "engagemind_user",
13     pwd: "engagemind_pass",
14     roles: [{ role: "readWrite", db: "engagemind_db" }]
15 })
16
17 # Update connection string in Engagemind-backend/.env
18 MONGO_URI=mongodb://localhost:27017/engagemindbackend
```

2.2.3 Starting Services

Start all three components:

Flask Backend:

```
1 cd EngageMind-rag
2 source venv/bin/activate # macOS/Linux
3 flask run --host=0.0.0.0 --port=5000
```

Express Authentication:

```
1 cd EngageMind-backend
2 npm run start
```


React Frontend:

```
1 cd EngageMind-frontend
2 npm run start
```

Access the application at <http://localhost:3000>

2.3 User Interaction with the EngageMind RAG Application

2.3.1 Introduction to the Frontend Interface

The frontend, accessible via a web browser, serves as the primary point of interaction, enabling users to register accounts, upload documents, and query the RAG system for contextually relevant responses. This section provides a comprehensive guide to navigating the frontend, detailing each step of the user journey from account creation to troubleshooting common issues.

- **User Registration:** Creating and verifying an account to access the system.
- **Logging In:** Authenticating users to enable personalized interactions.
- **Uploading Documents:** Enriching the RAG system with user-provided data.
- **Chatting with the System:** Querying the RAG module for document-based insights.
- **Troubleshooting:** Addressing common technical challenges.

The following subsections elaborate on each component, providing step-by-step instructions, example interactions, and solutions to potential errors, ensuring a seamless user experience.

2.3.2 User Registration

The first step for users engaging with the EngageMind RAG application is to create an account through the registration process. This section outlines the steps to register, verify an account, and handle common errors, ensuring users can seamlessly access the

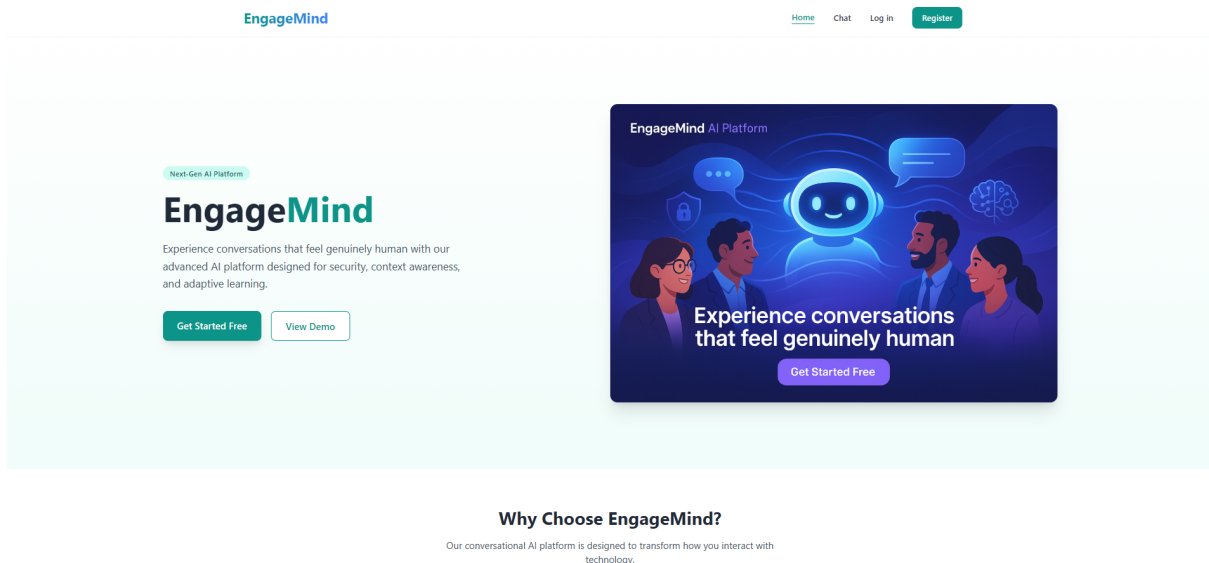


Figure 2.1: The landing page of the EngageMind RAG application, displaying the initial interface with prominent “Register” and “Login” buttons to guide user interaction.

system. The registration interface is designed to be intuitive, with real-time feedback to guide users through input validation and account activation.

Step-by-Step Registration Process

1. **Navigate to the Registration Page:** On the application’s landing page, users can locate the “Register” or “Sign Up” button, typically positioned in the top navigation bar or as a prominent call-to-action. Alternatively, the registration page is accessible directly via the URL `http://localhost:3000/register`.

2. Complete the Registration Form:

- **Input Fields:** Users must provide a valid email address, a secure password (minimum eight characters, including letters, numbers, and at least one special character), and, if required, additional details such as a username or full name.

- **Example Form:**

Email: `user@example.com`

Password: `*****`

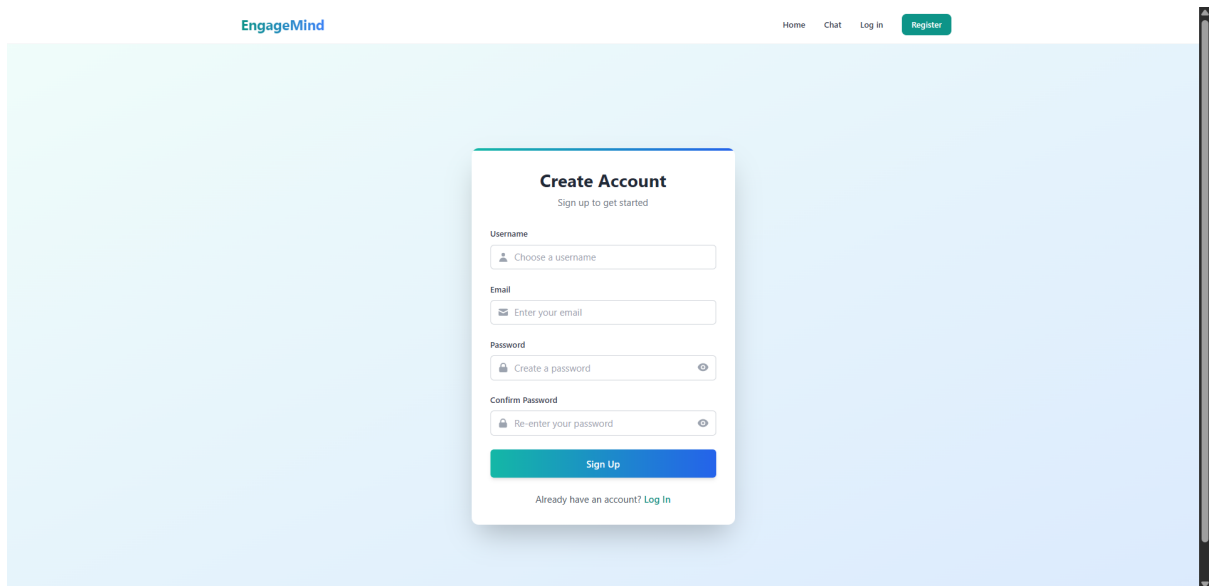


Figure 2.2: Register Page

Full Name: John Doe

- **Validation Feedback:** The form offers real-time validation. For instance, an incorrect email format triggers a message such as “Please enter a valid email address.”
3. **Submit the Form:** After filling in the required fields, users click the “Submit” or “Register” button to process their account creation.
 4. **Verify Email Address:**
 - Upon successful submission, the system sends a verification email containing an activation link to the provided email address.
 - Users must open the email and click the link to activate their account.
 - If the email does not appear in the inbox, users should check the spam folder or request a resend through the registration confirmation page.

Handling Registration Errors

Common issues during registration include duplicate email addresses or invalid inputs. The system provides clear error messages to guide users:

- **Duplicate Email:** If the error “Email already exists” appears, users should try a different email or use the “Forgot Password” option to recover an existing account.
- **Invalid Input:** Errors like “Invalid email format” or “Password too weak” prompt users to correct their entries based on the provided guidelines.

This registration process ensures secure and efficient onboarding, laying the foundation for subsequent interactions with the EngageMind application.

2.3.3 Logging In

After successfully registering and verifying their account, users can log into the EngageMind RAG application to access its features. This section details the login process, including how to navigate to the login page, use either standard or Google authentication, and reset forgotten passwords—ensuring a smooth authentication experience.

Step-by-Step Login Process

1. **Navigate to the Login Page:** From the landing page, users can click the “Login” button, typically located in the header or as a prominent option. Alternatively, the login page is accessible directly at <http://localhost:3000/login>.
2. **Login Options:** Users are provided with two methods to log into their account:
 - **Standard Login:** Enter the registered email address and password into the respective input fields.
 - **Google Login:** Click the “Sign in with Google” button to authenticate via Google. A secure OAuth2 flow will handle user validation, and upon success, the user is redirected to their dashboard.

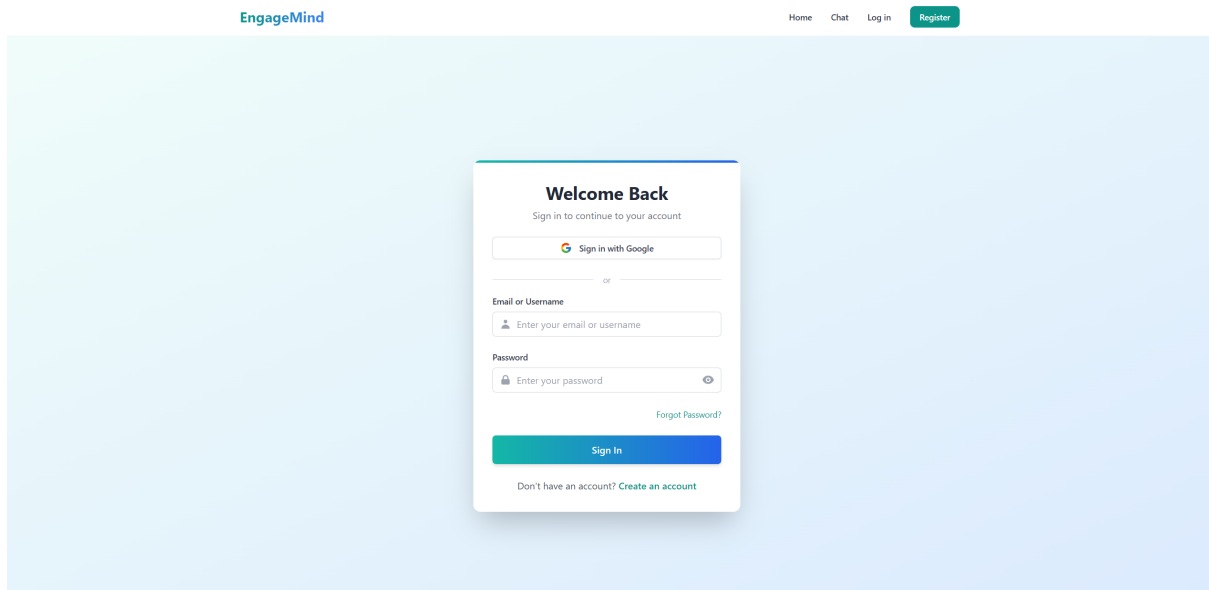


Figure 2.3: The login page of the EngageMind RAG application, featuring email/password login, Google sign-in, and password recovery options.

3. Enter Credentials (Standard Login):

- **Input Fields:**

Email: `user@example.com`

Password: `*****`

- **Submit:** Click the “Login” button to authenticate.

4. Troubleshoot Invalid Logins:

- **Error Handling:** If an error such as “Incorrect email or password” appears, users should check for typos or incorrect casing in their input.
- **Case Sensitivity:** Passwords are case-sensitive; ensure that Caps Lock is disabled.

5. Password Recovery:

- A “Forgot Password?” link is provided below the login form.
- Clicking this link takes users to the password recovery page, where they can enter their registered email address.

- An email will be sent to the provided address with a secure link to reset the password.
- After setting a new password, users can return to the login page and authenticate with their updated credentials.

The login process is designed to be user-friendly and secure, offering both standard and federated login options along with easy password recovery. Once authenticated, users are directed to their personalized dashboard to interact with the EngageMind RAG system.

2.3.4 Chatting with the RAG System

The chat interface is the core feature of the EngageMind RAG application, enabling users to interact dynamically with the system to query documents or ask general questions. Registered users can upload documents during a chat session, allowing the RAG (Retrieval-Augmented Generation) system to provide precise, context-aware answers. If no documents are uploaded, the system seamlessly performs a real-time web search using Tavily to generate relevant responses. This section outlines how users can access the chat interface, upload files, enter queries, and interact with the system effectively.

Accessing the Chat Interface

Only registered users can use the chat feature. After logging in (as described in Section 2.3.3), users are directed to their dashboard, from which they can access the chat interface by clicking the “Chat” option in the navigation menu.

Interacting with the Chat Interface

Once in the chat interface, users can begin conversing with the EngageMind system as follows:

1. Upload Documents (Optional):

- **Access Upload Feature:** Click the “+” icon (or paperclip symbol) adjacent to the chat input field to initiate document upload.

- **Select and Upload:** Choose a file from your device (.pdf, .txt, or image formats supported) and click “Upload.”
- **System Feedback:** Upon successful upload, a message like “Document uploaded successfully” appears, and the file is indexed for contextual response generation.
- **Error Handling:** For errors such as unsupported file types or size limits, refer to Section ??.

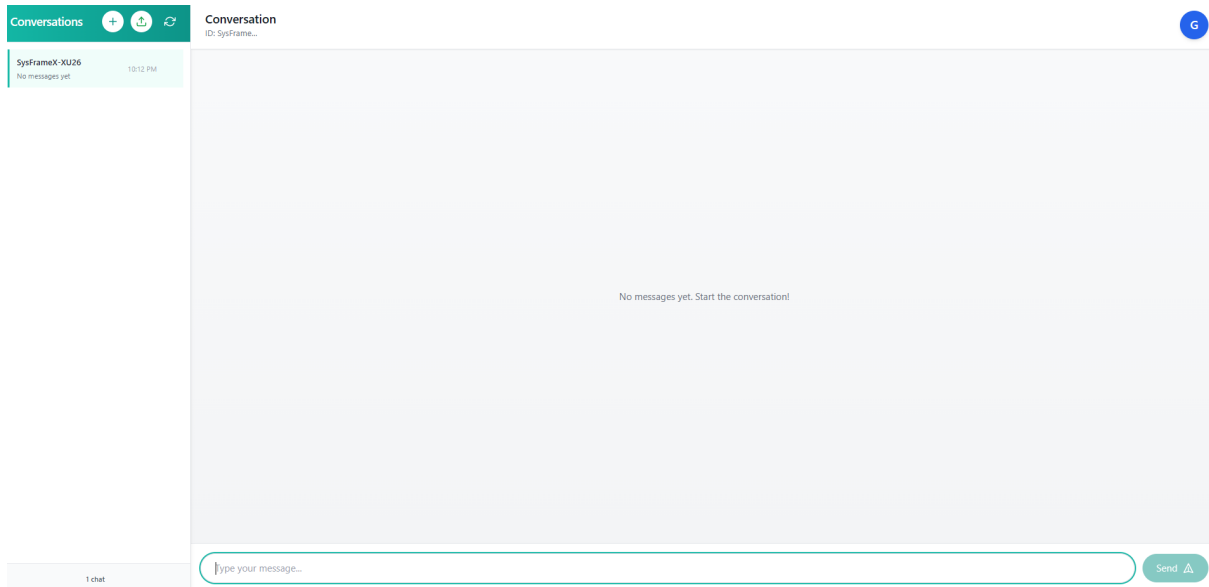


Figure 2.4: The EngageMind chat interface with conversation history, input field, and upload icon.

2. Enter a Query:

- **Text Input:** Type your question into the chat input field. For example:

Can you summarize the key ideas in the uploaded document?
- **Context Awareness:**
 - If a document is uploaded, the system retrieves relevant content from that file using the RAG pipeline.
 - If no document is uploaded, the system performs a real-time web search using Tavily to gather context and respond accordingly.

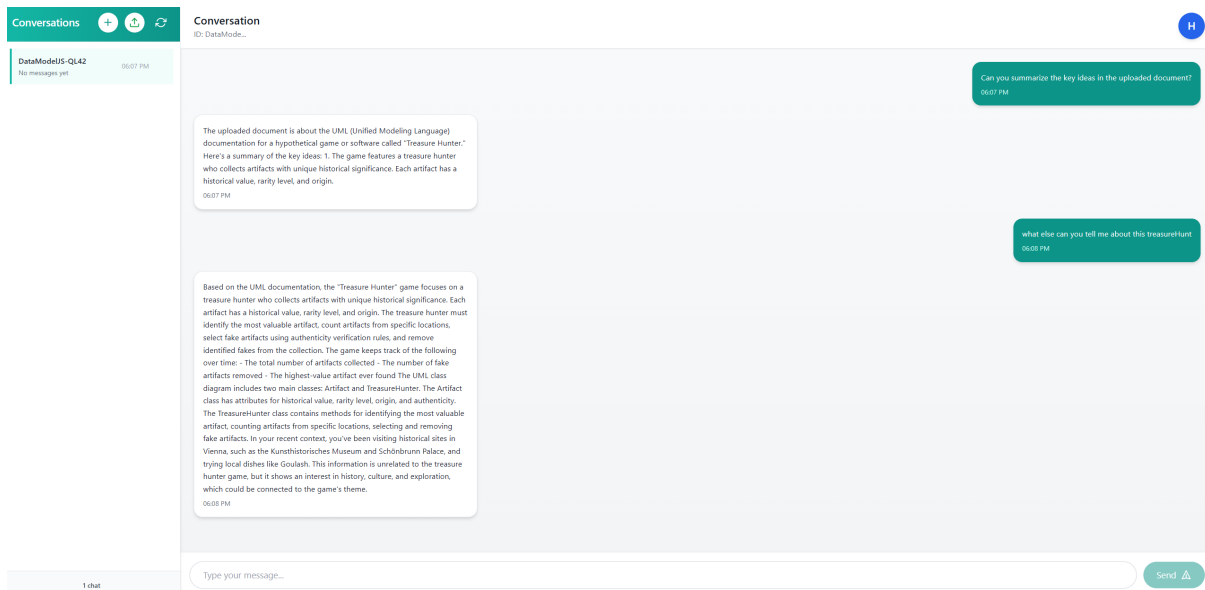


Figure 2.5: Example query entered in the chat interface.

3. Review System Responses:

- **Example (Document-Based):**

- **User:** “What are the main challenges discussed in the file?”
- **System:** “The main challenges discussed in the file revolve around the tasks of a treasure hunter who explores ancient ruins and collects artifacts. These challenges include: 1. Efficiently finding and organizing artifacts. 2. Identifying the most valuable artifact in their collection. 3. Counting how many artifacts originate from a specific location. 4. Selecting fake artifacts using the authenticity verification rules. 5. Removing fake artifacts from the collection, etc.”

- **Example (Web-Based):**

- **User:** “Who invented transformers in AI?”
- **System:** “The transformer model in AI, widely used in natural language processing, was introduced in the paper *Attention is All You Need* by Vaswani et al. (2017). Key contributors included Ashish Vaswani, Noam Shazeer, Niki Parmar, and others from Google Brain.”

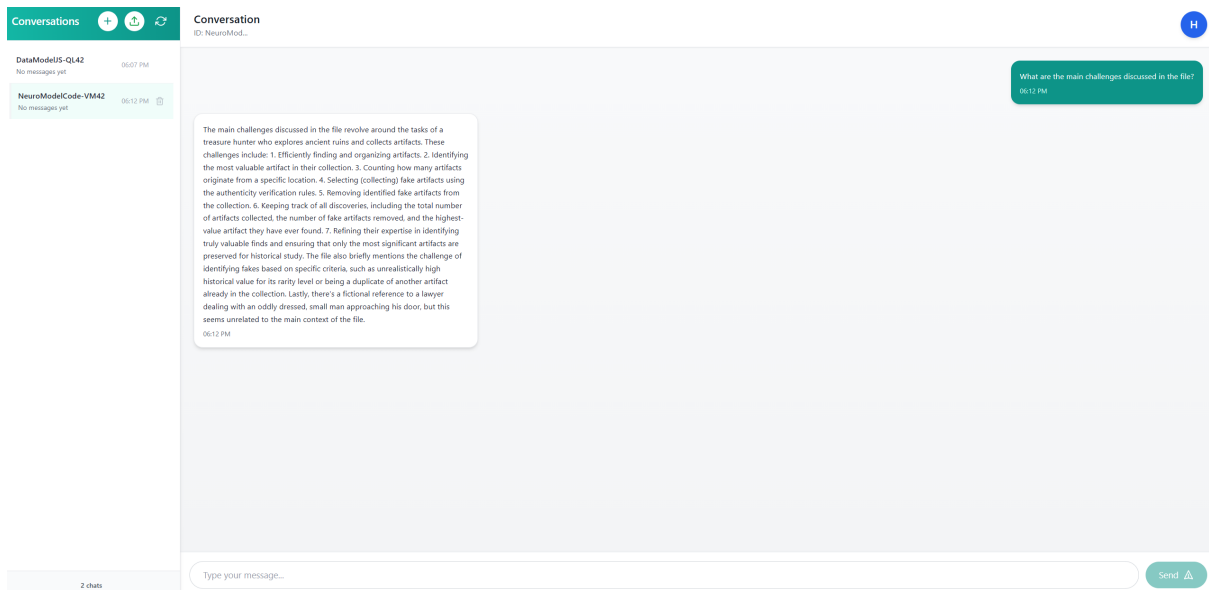


Figure 2.6: System response based on the uploaded document.

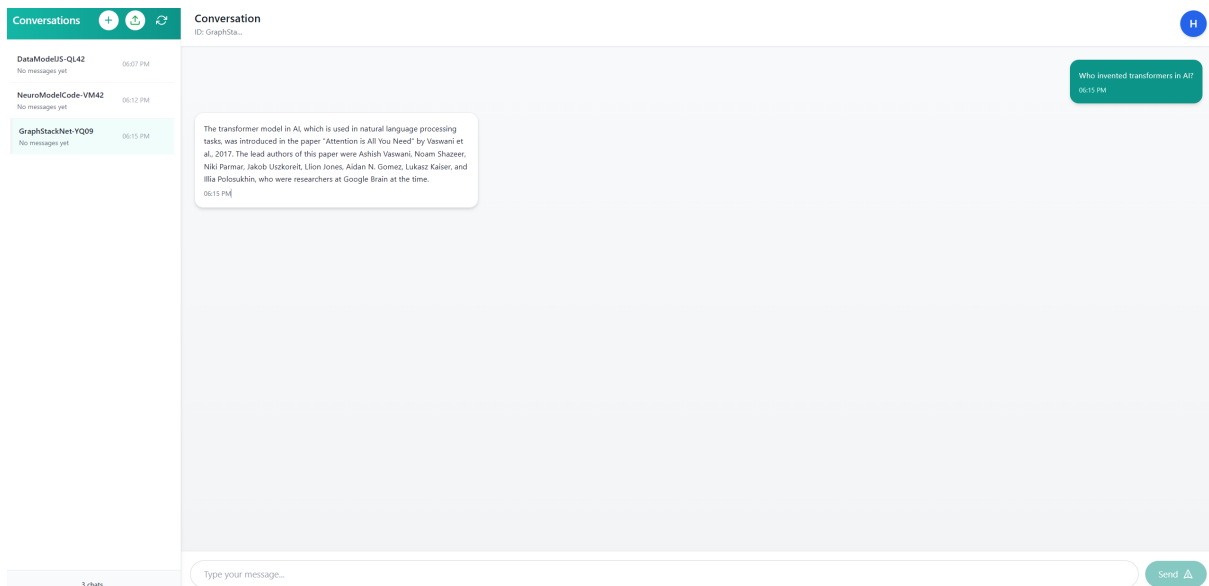


Figure 2.7: System response using Tavily web search when no document is uploaded.

4. Refine and Follow Up:

- Ask clarifying or follow-up questions for deeper insights.
- Maintain a clear, conversational tone for best results.

The chat interface provides a powerful and responsive user experience. Whether interacting with uploaded documents or relying on real-time web search, EngageMind ensures

accurate, relevant, and context-aware responses for each query.

2.3.5 Troubleshooting

When interacting with the EngageMind RAG application, users may encounter common technical issues. This section provides practical solutions to typical problems, ensuring a smooth experience across different operating systems. Issues such as port conflicts, missing dependencies, CORS errors, and frontend loading failures are addressed with step-by-step resolutions.

Common Issues and Solutions

- **Port Conflicts:**

- **Symptom:** The server fails to start because the port is already in use.
- **Solution:**
 - * Change the port number in the configuration file (e.g., server.js for the backend).
 - * Alternatively, free the port by terminating the process:

- **Linux/macOS:**

```
1 lsof -i :3000
2 kill -9 <PID>
```

- **Windows:**

```
1 netstat -ano | findstr :3000
2 taskkill /PID <PID> /F
```

- **Missing Dependencies:**

- **Symptom:** Errors indicating missing packages during startup.
- **Solution:**
 - * For Node.js components, reinstall dependencies:

```
1 npm install
```

- * For Python dependencies in the RAG component:

```
1 pip install -r requirements.txt
```

- **CORS Issues:**

- **Symptom:** The frontend cannot fetch data from the backend, resulting in CORS error messages in the browser console.
- **Solution:** Adjust CORS settings in the backend Express server:

```
1 const cors = require('cors');  
2 app.use(cors({ origin: 'http://localhost:3000' }));
```

- **Frontend Does Not Load:**

- **Symptom:** The browser displays a blank page or fails to render the application.
- **Solution:**
 - * Check the console log for build errors.
 - * Clear the browser cache.
 - * Ensure both the backend and RAG components are running.

OS-Specific Notes

- **Windows:**

- Run terminals as Administrator if required.
- Ensure firewall or antivirus software does not block Node.js or Python processes.

- **Linux/macOS:**

- Use sudo for installation commands if permission issues arise.
- Adjust file permissions if errors occur.

This troubleshooting guide equips users with the tools to resolve common issues, ensuring the EngageMind RAG application operates smoothly across diverse environments.

2.3.6 Example Commands

To assist users in understanding the setup and operation of the EngageMind RAG application, this section provides example terminal commands for configuring the RAG component and verifying the backend server's status. Screenshots of the application's interfaces have been included after each relevant subsection (e.g., Figures 2.2, 2.3, 2.4) to visually demonstrate the user experience.

Example Terminal Commands

The following commands illustrate how to set up the RAG component on different operating systems and perform an API health check.

- **Linux/macOS RAG Setup:**

```
1 # Navigate to the 'rag' folder
2 cd ~/Thesis/rag
3
4 # Create and activate a Python virtual environment
5 python3 -m venv venv
6 source venv/bin/activate
7
8 # Install dependencies from requirements.txt
9 pip install -r requirements.txt
10
11 # Start the RAG component
12 python run.py
```

- **Windows RAG Setup:**

```
1 REM Navigate to the 'rag' folder
2 cd C:\Users\Desktop\Thesis\rag
3
4 REM Create and activate a Python virtual environment
5 python -m venv venv
```

```
6 venv\Scripts\activate
7
8 REM Install dependencies
9 pip install -r requirements.txt
10
11 REM Start the RAG component
12 python run.py
```

- **API Health Check:** To verify the backend server is operational, users can run:

```
1 curl http://localhost:3000/api/health
```

This command returns a JSON response (e.g., { "status": "ok" }) indicating the API is live.

These commands provide a practical reference for setting up and verifying the application's components, complementing the visual aids provided throughout this guide.

Chapter 3

Developer Documentation

Chapter 4

Developer Documentation

4.1 Cloning the Repository

Cloning the EngageMind repository is the first critical step for any developer joining the project. This section outlines a comprehensive guide to obtaining the full source code, ensuring that all modules, configuration files, and documentation are available on your local machine.

4.1.1 Prerequisites for Cloning

Before cloning, ensure that Git is installed on your development machine:

- **Windows:**

- The EngageMind repository is hosted on GitHub [2], and requires Git [3] to be installed.
- Open a Command Prompt or Git Bash to verify installation by running:

```
1 git --version
```

- **macOS:**

- macOS users may opt to use Homebrew [4] to install Git or other dependencies.
- Verify by running in the Terminal:

```
1 git --version
```

- **Linux (Ubuntu/Debian):**

- Update your package lists and install Git:

```
1 sudo apt-get update
2 sudo apt-get install git -y
```

- Confirm the installation with:

```
1 git --version
```

4.1.2 Repository Details

- **Repository URL:**

The EngageMind repository is hosted on GitHub at:

<https://github.com/haseebraza715/EngageMind.git>

- **Branching Strategy:**

The primary branch (commonly named **main** or **master**) contains the stable version of the codebase. Additional branches may exist for feature development or experimental changes. Always verify with your team which branch to work on.

- **Tags and Releases:**

Releases follow a semantic versioning standard (e.g., **v1.0.0**) for referencing stable builds, comparing changes, or rollback.

4.1.3 Cloning the Repository

Follow these steps to clone the EngageMind repository:

1. **Open Terminal/Command Prompt:**

Navigate to your preferred directory.

2. **Clone the Repository:**

```
1 git clone https://github.com/haseebraza715/EngageMind.git
```

This command creates a folder **EngageMind** with the project files.

3. **Enter the Repository Directory:**


```
1 cd EngageMind
```

4.1.4 Verifying the Local Repository

– **Check Git Status:**

```
1 git status
```

Should show a clean working directory.

– **Review Remote Settings:**

```
1 git remote -v
```

Ensures the origin points to the EngageMind GitHub repository.

4.1.5 Repository Structure Overview

Understanding the directory structure is essential:

- `/EngageMind-frontend`: React app with UI components, services, and assets.
- `/EngageMind-backend`: Express service for authentication, sessions, and APIs.
- `/EngageMind-rag`: Flask service for GPT-2 and RAG-based learning.

4.1.6 Handling Repository Updates

To update your local repository:

```
1 git pull origin main
```

Fetches and merges the latest commits from the `main` branch.

4.1.7 Managing Submodules (If Applicable)

If the repository uses submodules:

1. **Initialize Submodules:**

```
1 git submodule update --init --recursive
```

Ensures submodules are downloaded and initialized.

4.1.8 Troubleshooting Common Issues

- **Permission and Network Issues:**

Verify your network or switch Git protocols (SSH/HTTPS).

- **Authentication Errors:**

Configure SSH keys or personal access tokens for GitHub.

- **Corrupted Cloning:**

Delete the folder and repeat the cloning process.

4.2 Prerequisites

This section outlines the necessary technologies, dependencies, and configurations required to run and develop the EngageMind system.

4.2.1 MongoDB (Database)

EngageMind uses **MongoDB** [5] as the primary data store to persist user profiles, documents, memory records, feedback, and conversations.

Setup Instructions

You can either:

- Install MongoDB locally: [MongoDB Community Edition](#)
- Use a managed cluster via [MongoDB Atlas](#)

Environment Configuration

In your `.env` file for both the Flask and Express services, add:

```
1 MONGO_URI=mongodb://localhost:27017/engagemind
```

Replace the URI if you're using a hosted instance.

MongoDB Collections

The following collections are used:

Collection	Description
users	Stores auth credentials, profile info
documents	Uploaded file metadata and contents
conversations	Message history per user
memories	Persistent memory for each user session

4.2.2 Flask Backend (RAG + Inference Engine)

The **Flask-based Python backend** [6] handles ingestion, retrieval, memory, and response generation using LangChain, LangGraph, FAISS, and GPT/Mistral APIs.

Python Version

- Python 3.10 or higher recommended.

Installing Dependencies

Navigate to the backend directory and create a virtual environment:

```
1 python -m venv venv
2 source venv/bin/activate # On Windows: venv\Scripts\activate
3 pip install -r requirements.txt
```

This installs:

- Flask (API server)
- LangChain (retrieval) [7]
- LangGraph (memory) [8]
- FAISS (vector similarity) [9]
- PyTorch (for GPT-2 fine-tuning) [10]

Key Features

- **Document ingestion pipeline:** Accepts multiple file types and chunks them for vector storage.
- **LangChain integration:** Handles context-aware RAG with memory recall.
- **FAISS search engine:** Enables fast similarity queries over embedded data.
- **Real-time communication:** Socket.IO allows users to receive responses instantly.

4.2.3 Express Backend (User Authentication)

The **Express backend** [11] is responsible for secure authentication and profile handling.

Node Requirements

- Node.js v18+ and npm. Install from Node.js official site. [12]

Installing Dependencies

Inside the `engagemind-backend/` directory:

```
1 npm install
```

This sets up:

- Express (server)
- Passport.js (OAuth strategies) [13]
- JWT (token management) [14]
- Resend (email API) [15]

Auth Features

- **JWT Auth:** Email/password registration and login flow with secure tokens.
- **Google OAuth:** Integrated via Passport.js, handles social sign-ins. [16]

- **Resend Email API:**
 - Sends verification emails after registration.
 - Provides password reset via emailed links.
- **Profile APIs:** Allow users to edit name, avatar, bio, and view history.

4.2.4 React Frontend

The frontend is built using **React** [17] (with either **Vite** [18] or Create React App) and styled using **Tailwind CSS** [19].

Features

- **Authentication UI:**
 - Login and signup pages.
 - Google OAuth button.
 - Forgot password + Resend verification.
- **Chat Interface:**
 - Input + response threads.
 - File uploader.
 - Feedback controls.
- **Settings Page:**
 - Edit profile info (name, bio, avatar).
 - View usage history.
- **Error Boundaries:**
 - Graceful fallback UIs for route access errors and missing data.

Setup Instructions

Navigate to the frontend folder and run:

```
1 npm install
2 npm run dev
```

Use `.env` to define API endpoints for the backend.

4.2.5 Background Services: Celery and Redis

These services enable asynchronous document handling and real-time feedback.

Redis

Redis [20] is used as a **message broker** between Flask and Celery.

- Required for task queue management. Install via:

```
1 sudo apt install redis-server
```

- Running Redis locally on port 6379.

Celery

Celery [21] handles asynchronous jobs like:

- Embedding document chunks
- Storing feedback
- Monitoring long-running tasks

Start the worker inside the Flask directory with:

```
1 celery -A app.celery worker --loglevel=info
```

Ensure Redis is running before launching the Celery worker.

Following these setup steps ensures all core EngageMind services are properly initialized for development and testing.

4.3 External APIs Used

EngageMind integrates several external APIs to enhance core functionalities such as AI-powered responses, intelligent search, email delivery, and social login. This section documents the APIs in use, their roles, authentication setup, and integration patterns.

4.3.1 Mistral API

Features:

- High-quality completions via hosted LLM
- Stateless, cost-effective inference layer

Setup:

- Register at the Mistral API Platform [\[22\]](#).
- Generate your API key and store in `.env`:

```
1 MISTRAL_API_KEY=your_mistral_api_key
```

Integration:

- Used in the Flask backend via HTTP requests.
- Activated when:
 - No relevant documents are retrieved from FAISS.
- Sample usage (Python):

```
1 headers = {"Authorization": f"Bearer {MISTRAL_API_KEY}"}
2 response = requests.post(MISTRAL_URL, headers=headers, json={"prompt": prompt})
```

4.3.2 Tavily Web Search API

Purpose: Provides up-to-date, real-world search results to improve RAG by retrieving external context.

Features:

- Retrieves summaries of relevant web articles
- Used when internal document memory is insufficient

Setup:

- Sign up at Tavily API Platform [\[23\]](#).
- Add to your environment variables:

```
1 TAVILY_API_KEY=your_tavily_api_key
```

Integration:

- Invoked from the Flask backend in the retrieval pipeline.
- Fallback after FAISS check.
- Search results are embedded using LangChain and added temporarily to the memory window.

4.3.3 Resend API (Email Delivery)

Purpose: Handles transactional emails such as:

- Verification email on registration
- Password reset emails

Features:

- Fast, reliable email delivery
- HTML templating support

- Dashboard for analytics

Setup:

- Sign up at Resend Email API Service [\[15\]](#).
- Add to your `.env` in the Express auth service:

```
1 RESEND_API_KEY=your_resend_api_key
2 RESEND_FROM_EMAIL=noreply@yourdomain.com
```

Integration:

- Password reset:
 - Generates a secure link with a token.
 - Sends it using Resend's transactional email API.
- Email verification:
 - Triggered after user registration.
 - Token is saved in DB and validated on confirmation.
- Sample call (Node.js):

```
1 await resend.emails.send({
2   from: process.env.RESEND_FROM_EMAIL,
3   to: user.email,
4   subject: "Reset your password",
5   html: '<a href="${resetUrl}">Click here to reset</a>'
6 });
```

4.3.4 Google OAuth (Passport.js)

Purpose: Allows users to sign up and log in using their Google account.

Features:

- One-click login
- Auto-fill profile details
- Secure token exchange via Google

Setup:

- Create OAuth credentials via Google Cloud Console [24].
- Add the following to your `.env` in the auth server:

```
1 GOOGLE_CLIENT_ID=your_google_client_id
2 GOOGLE_CLIENT_SECRET=your_google_client_secret
```

Integration:

- Passport.js strategy configured for Google.
- Tokens exchanged and user stored or retrieved from DB.
- Integrated into frontend login flow with a Google button.

4.4 Software Architecture

4.4.1 System Overview

EngageMind is a robust, full-stack AI platform designed to deliver highly personalized and context-aware interactions through a retrieval-augmented generation (RAG) framework. It empowers users to upload multi-format documents, engage in dynamic conversations with AI agents, and manage secure profiles. Key capabilities include:

- Multi-format document ingestion with optical character recognition (OCR) support
- Stateful, context-sensitive conversations driven by a LangGraph workflow
- Persistent conversational memory and history
- Secure authentication supporting email/password and Google OAuth
- Feedback-driven response evaluation for hallucination and relevance

The architecture is modular, comprising three core services:

1. **Frontend (React)**: A responsive for user interaction
2. **Auth Backend (Node.js + Express)**: Handles authentication, user profiles, and email workflows
3. **RAG Backend (Python + Flask)**: Manages document ingestion, retrieval, and conversational AI logic

Persistent data is stored in **MongoDB**, while **FAISS** vector stores enable efficient semantic search, with indexes maintained per user for isolation.

4.4.2 Architecture Diagram

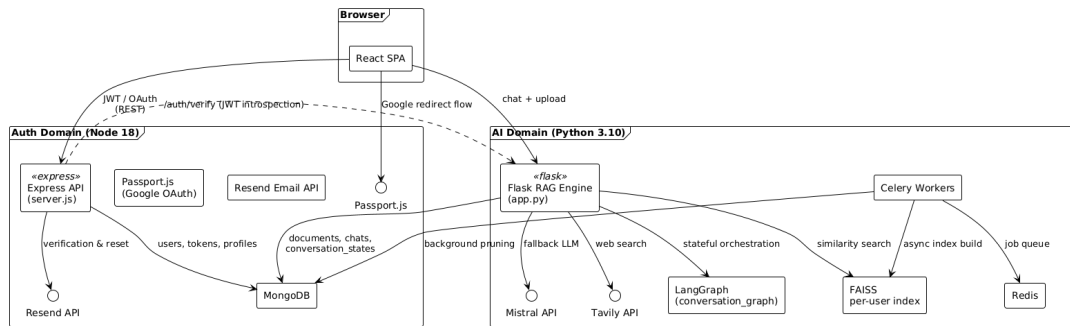


Figure 4.1: EngageMind System Architecture

4.4.3 System Logic Overview

EngageMind employs an event-driven architecture, integrating stateless RESTful APIs with a stateful **LangGraph** agent to manage conversational memory and dialogue flow. The primary workflows are:

- **Document Upload:**
 - Initiated via React frontend to Flask `/api/upload`
 - Flask processes the file, invoking `build_faiss_index` for OCR, embedding, and vector indexing
 - Stores metadata and content in MongoDB; saves FAISS index to disk

- **Chat Interaction:**

- Triggered via React to Flask `/api/conversation/:id/message`
- Flask retrieves conversation history and invokes LangGraph, which:
 1. Fetches relevant context using query rewriting and FAISS
 2. Generates a response via the RAG model
 3. Evaluates response for hallucination and relevance
 4. Persists updated memory and assistant message
- Returns response to frontend

- **Memory Management:**

- Conversation state stored in MongoDB (`conversation_states`)
- LangGraph manages state transitions using serializable Python dictionaries

4.4.4 RAG System Details

Document Ingestion (`ingestion_pipeline.py`)

The ingestion pipeline processes uploaded documents to enable semantic search:

- **Supported Formats:** Direct support for `.txt`; other formats (`.pdf`, `.jpg`) processed via **Mistral OCR API**
- **Chunking:** Text segmented using `RecursiveCharacterTextSplitter` for optimal embedding
- **Embedding:** Generates vectors with `MistralAIEmbeddings`
- **Indexing:** Constructs a FAISS vector store, saved to `faiss_index_dir/{user_id}` for per-user isolation

Retrieval Logic (`retrieval_pipeline.py`)

The retrieval pipeline ensures precise context retrieval:

- Loads user-specific FAISS index from disk
- Uses a **rewriting retriever** powered by LangChain, reformulating queries via a Mistral-based LLM prompt
- Returns top-k documents based on semantic similarity

4.4.5 LangGraph Memory Workflow (`app.py`)

Conversations are powered by a **LangGraph** workflow with two key nodes:

1. **retrieve__context:**

- Loads conversation history from MongoDB
- Rewrites the user query for enhanced retrieval
- Executes the retrieval chain to fetch relevant documents
- Updates the conversation's memory state

2. **generate__response:**

- Builds a prompt with context and chat history
- Invokes the Mistral LLM for response generation
- Performs hallucination and relevance grading using LangChain
- Appends a disclaimer if the response fails grounding or relevance checks
- Saves the updated conversation state

Memory Persistence:

- Stored in the `conversation_states` MongoDB collection
- State is a serialized Python `dict` containing:
 - `user_id`, message history, retrieved context (documents), and web fallback flag

4.4.6 Authentication System

Express + MongoDB (`server.js`, `userModel.js`)

The authentication service ensures secure user management:

- **Local Auth:** Email/password registration and login
- **Google OAuth:** Seamless social login via **Passport.js**
- **Email Workflows:** Verification and password reset emails sent via **Resend API**
- **Token Management:** JWTs issued upon login, stored client-side

MongoDB User Schema

```

1 {
2   email, password, provider: 'local' | 'google',
3   googleId, verified,
4   avatar, bio, socialLinks, role,
5   verificationToken, passwordResetToken
6 }
```

4.4.7 Database Design (MongoDB)

The MongoDB database is structured as follows:

Collection	Purpose
<code>users</code>	Authentication and profile information
<code>documents</code>	Ingested documents with binary content
<code>chats</code>	Chat sessions with all messages
<code>conversation_states</code>	Serialized LangGraph memory state

Table 4.1: MongoDB Collections

Each user has:

- A dedicated FAISS index stored on disk
- Multiple chat threads tracked in `chats`

- Per-message state persistence in `conversation_states`

4.4.8 API Surface

Flask `/api/`

Endpoint	Method	Description
<code>/api/upload</code>	POST	Uploads document, stores in MongoDB for indexing
<code>/api/conversation</code>	POST	Creates a new conversation thread
<code>/api/conversation/:id/message</code>	POST	Sends message, returns AI response
<code>/api/conversation/:id</code>	GET	Loads full conversation
<code>/api/conversations</code>	GET	Lists all user threads
<code>/api/conversation/:id</code>	DELETE	Deletes thread

Table 4.2: Flask API Endpoints

Express `/auth/`

Endpoint	Method	Description
<code>/auth/register</code>	POST	Registers user
<code>/auth/login</code>	POST	Logs in and returns JWT
<code>/auth/google</code>	GET	Google OAuth redirect
<code>/auth/reset</code>	POST	Sends password reset link
<code>/profile</code>	PUT	Updates name, bio, avatar

Table 4.3: Express API Endpoints

4.4.9 Real-Time & Evaluation

- **Response Delivery:** Assistant responses are delivered instantly via REST APIs, with WebSocket support planned for streaming
- **Evaluation:** LangChain-based grading assesses:
 - **Grounding:** Is the response supported by retrieved documents?
 - **Relevance:** Does the response address the user’s query?
- **Disclaimer:** Added to responses failing grounding or relevance checks

4.4.10 Deployment Considerations

- **Local Development:** Managed via flask and express backend, orchestrating `frontend`, `rag` backend, `user auth`, and `mongo` services
- **Storage:** FAISS indexes stored in directory under `faiss_indexdir` with per user indexes. **Stateless Design** : *All services are stateless except for MongoDB and FAISS index directories*
- **Rate Limiting:** Implemented using `Flask-Limiter` to prevent abuse
- **Configuration:** Environment variables stored in `.env` files for all services

4.5 RAG Implementation

The RAG (Retrieval-Augmented Generation) backend forms the core of EngageMind's personalized conversational AI system. It provides the ingestion of user-specific documents, retrieval of relevant content based on queries, and a Flask-based API server that connects the RAG system with the frontend.

4.5.1 Overview of RAG backend

The backend system is modularized into distinct components handling ingestion, retrieval, and server communication. It uses FAISS for high-speed similarity search and MongoDB to manage user-specific data. The system is designed to support multi-file ingestion, incremental updates, and efficient retrieval with minimal latency.

4.5.2 Ingestion Pipeline

Purpose

The ingestion pipeline in EngageMind serves as the foundation for enabling external knowledge retrieval. It is responsible for processing user-uploaded documents, extracting textual data, chunking content into manageable units, generating semantic embeddings, and storing them efficiently into a user-specific FAISS index. This allows the system to

later retrieve relevant information in response to user queries.

The ingestion pipeline is designed with modularity, robustness, and scalability in mind, supporting diverse file formats, caching, parallel processing, and tight integration with vector database storage.

Detailed Workflow

The ingestion process is comprised of multiple systematic steps:

1. File Validation and Caching

- Upon receiving a document (file path), the system first verifies if the file physically exists.
- An MD5 hash of the file's contents is computed (`get_file_hash(file_path)`), which serves as a unique fingerprint.
- If a document with the same hash has already been processed, the system retrieves cached results from `file_hash_cache` instead of reprocessing, greatly improving efficiency for duplicate uploads.

2. Content Extraction

- Depending on the file type:
 - **Plain Text Files (.txt):**
 - * Directly loaded using the `TextLoader` class (`process_text_file(file_path)`).
 - * The extracted content is wrapped into LangChain's `Document` objects, attaching user-specific metadata.
 - **Non-Text Files (e.g., PDFs, Images):**
 - * Processed via Optical Character Recognition (OCR) using the Mistral API.
 - * `run_ocr_with_mistral(client, file_path)` uploads the file, generates a signed URL, and invokes the Mistral OCR service to extract text.

- * A retry mechanism (`run_ocr_with_retry`) ensures robustness against transient failures (up to two retries with delays).
- * OCR output is wrapped into `Document` objects with appropriate metadata.
- This dual-pathway ensures the ingestion system can handle both structured (text) and unstructured (image-based) documents.

3. Parallel Processing

- If multiple files are uploaded simultaneously, the system employs a `ThreadPoolExecutor` to process files concurrently, significantly speeding up the ingestion phase (`max_workers` configurable).
- Each file is assigned to a separate thread for independent loading and text extraction.

4. Chunking

- Once documents are extracted, they are passed to a `RecursiveCharacterTextSplitter`.
- The text is split into coherent chunks of approximately 1000 characters, with an overlap of 100 characters between adjacent chunks.
- This technique ensures that semantic coherence is preserved across chunks, improving embedding quality and retrieval accuracy.

5. Embedding Generation

- The textual chunks are then embedded using the `MistralAIEmbeddings` model via the `MistralAIEmbeddings` class.
- Each chunk is transformed into a high-dimensional vector that captures its semantic meaning.
- **Embedding Model:**
 - *Model Used:* "mistral-embed"

- *API Key Authentication:* Secure access via environment-configured `MISTRAL_API_KEY`.

6. FAISS Index Construction

- A FAISS (Facebook AI Similarity Search) vector store is created using the generated embeddings.
- FAISS provides highly efficient approximate nearest neighbor (ANN) search, enabling real-time retrieval performance even for large-scale datasets.

7. Index Saving

- After constructing the vector store:
- The FAISS index (vectors + metadata) is saved locally under a user-specific directory at `FAISS_INDEX_ROOT/{user_id}`.
- Two files are created:
 - `index.faiss` (vector index)
 - `index.pkl` (document metadata)
- This ensures persistence of user knowledge bases across sessions, enabling future retrieval without requiring re-ingestion.

MongoDB Integration (Optional Ingestion Path)

In addition to direct file uploads, EngageMind supports ingestion from a MongoDB collection:

- Documents associated with a user are fetched from the database.
- Plain text files are decoded directly; non-text binaries are processed via temporary file storage and OCR extraction.
- This design allows seamless ingestion from both local file systems and cloud-stored documents.

- Main function handling this:
 - `load_documents_from_mongo(mongo_collection, user_id)`

Error Handling and Robustness

Several layers of error handling are incorporated to make the ingestion process production-ready:

- Retry mechanisms for unstable OCR operations.
- Thread-safe operations using local clients.
- Detailed logging for success, warnings, and exceptions (`logger_setup.py`).
- File cleanup for temporary files after OCR processing to avoid storage bloating.

Key Functions and Their Roles

Function	Role
<code>process_file(file_path, user_id, api_key)</code>	Main dispatcher to handle different file types and apply caching
<code>run_ocr_with_retry(client, file_path)</code>	Robust OCR invocation with automatic retry logic
<code>build_faiss_index(user_id, input_files, api_key, mongo_collection)</code>	Orchestrates the full ingestion workflow: loading, chunking, embedding, saving
<code>query_faiss_index(vector_store, query)</code>	Enables similarity search over the constructed FAISS index

Table 4.4: Key Functions in Ingestion Pipeline

4.5.3 Retrieval Pipeline

Purpose

The retrieval pipeline in EngageMind is responsible for fetching the most relevant document chunks from the user’s personalized knowledge base in response to a user query.

It dynamically enhances the model's generation ability by grounding its responses with contextually appropriate and user-specific external knowledge.

The retrieval pipeline is tightly integrated with the ingestion module, operating on the FAISS indices previously built for each user. Additionally, it features query rewriting capabilities to improve the quality of document retrieval in multi-turn conversational settings.

Detailed Workflow

The retrieval process consists of the following major steps:

1. Loading the FAISS Index

- Upon receiving a query, the system first loads the user-specific FAISS index from disk.
- The location is determined based on the `user_id`, under the root directory defined by `FAISS_INDEX_ROOT`.
- If the directory or index does not exist, an error is raised, instructing the user to upload documents first.
- Key components:
 - `MistralAIEmbeddings`: Used for embedding new queries.
 - `FAISS.load_local()`: Deserializes the stored index along with document meta-data.
- This ensures that each user accesses only their own personalized document space.

2. Building the Base Retriever

- A base retriever is instantiated from the loaded FAISS vector store.
- This retriever performs similarity search over the document embeddings using vector distance measures (e.g., cosine similarity or inner product).

- Retrieval is configured to return the top-k most relevant chunks (k=4 by default), filtered by `user_id` to ensure user-specific isolation.
- Key method:
 - `vector_store.as_retriever(search_kwargs={"k": k, "filter": {"user_id": user_id}})`

3. Enhancing Retrieval with Query Rewriting

- To improve retrieval in conversational settings where user queries may be elliptical, short, or context-dependent, a query rewriting mechanism is introduced.
- A `RewritingRetriever` class wraps around the base retriever.
- How it works:
 - When a query is received:
 - * It is first rewritten by an LLM (`ChatMistralAI`) using the `rewrite_query` function (defined in the evaluation module).
 - * The rewritten query is then passed to the base retriever to perform the similarity search.
 - This improves the relevance and accuracy of the documents retrieved, especially when user queries depend heavily on previous conversation history.

4. Building the Retrieval-Augmented Chain

- After obtaining the relevant documents, the system constructs a retrieval-augmented generation chain (`retrieval_chain`).
- Components involved:
 - **Prompt Template:**
 - * A `ChatPromptTemplate` is used to define the structure of inputs to the LLM.
 - * It injects the following into the model's prompt:
 - `<conversation_history>`: (optional) Past chat history.

- `<context>`: Retrieved document chunks.
- `Question`: The user's input.
- **Stuff Document Chain:**
 - * The retrieved documents are "stuffed" (combined) into a single prompt to the model.
- **Final Chain Construction:**
 - * The `retrieval_chain` is created by connecting:
 - The `RewritingRetriever` (enhanced retriever) and
 - The `StuffDocumentChain` (combining retriever results and generating the answer).
- Thus, the final retrieval pipeline ensures that every user query is processed intelligently:
 - By rewriting it for maximum clarity,
 - Fetching the best contextual documents, and
 - Answering based on personalized external knowledge.

Error Handling

- **Missing Index Handling:**
 - If the user has not ingested any documents, the pipeline gracefully informs the user to upload documents before asking questions.
- **Authentication Checks:**
 - The system ensures that an API key (`MISTRAL_API_KEY`) is available before attempting embeddings or LLM operations.
- **Logging:**

- The system logs key steps such as index loading, retrieval chain creation, and any errors encountered for troubleshooting and system monitoring.

Key Classes and Functions

Component	Description
<code>RewritingRetriever</code>	A custom retriever class that rewrites queries before retrieval
<code>rewrite_query(query, llm)</code>	A function that refines the user's input query based on conversation context
<code>build_retrieval_chain(user_id, api_key, embed_model_name, index_root, k)</code>	Main function to build and return the full retrieval chain for a given user
<code>ChatMistralAI</code>	The LLM used for both query rewriting and final response generation

Table 4.5: Key Components in Retrieval Pipeline

Advantages of the Retrieval Pipeline Design

- **Personalization:**
 - Each user has a separate FAISS index, ensuring privacy and tailored responses.
- **Scalability:**
 - By dynamically loading FAISS indices per user and performing lightweight retrieval, the system scales to handle many concurrent users.
- **Improved Recall:**
 - Query rewriting boosts the probability that relevant documents are retrieved, particularly for follow-up questions in multi-turn conversations.
- **Modularity:**
 - Retrieval, rewriting, and generation are cleanly separated, making it easy to replace components (e.g., switching LLM models or retrieval methods).

4.5.4 Server and API Setup

Purpose

The Server and API layer in EngageMind provides the bridge between the frontend user interface and the backend functionalities (retrieval, ingestion, conversation management). It exposes a set of secured REST API endpoints built with Flask that allow users to upload documents, initiate conversations, send questions, and retrieve answers augmented with personalized document knowledge.

This layer ensures smooth interaction, authentication, session management, conversation state persistence, and fallback handling when retrieval fails.

System Overview

- **Framework:** Flask (Python web microframework)
- **Database:** MongoDB (via PyMongo)
- **Vector Store:** FAISS (for document retrieval)
- **Third-Party APIs:**
 - Mistral AI for embeddings and LLM responses
 - Tavily API for Web Search fallback (if no documents found)
- **Authentication:** JWT token-based authentication (via `login_required`)
- **Security:** CORS settings and Rate Limiting
- **State Management:** MongoDB-backed conversation state saving and loading
- **Conversation Control:** LangGraph State Graph for conversational workflow

Detailed Server Architecture

1. API Server Initialization

- The API server is initialized by the `create_app()` function which:

- Configures CORS based on environment settings.
- Connects to MongoDB and sets up necessary indices for faster query performance.
- Initializes rate limiting using Flask-Limiter, protecting against abuse.
- Registers all API routes via the `register_routes()` function.
- Logging is set at different levels depending on the environment (development or production).

2. Authentication and Security

- All API endpoints under `/api/` are protected with a JWT token-based authentication layer:
 - Middleware (`login_required`) validates the token and attaches the authenticated user's ID (`g.user_id`) to the request context.
 - Rate Limiting (default 100 requests/minute, 2000 requests/hour) ensures fair use.
 - Security mechanisms are extendable to include OAuth2 or API key-based systems if required in the future.

3. Core API Endpoints

Endpoint	Description
POST <code>/api/upload</code>	Uploads a document and rebuilds the user's FAISS index
POST <code>/api/ask</code>	Allows the user to send a question and retrieve an answer using RAG
POST <code>/api/conversation</code>	Creates a new conversation session with a unique ID
POST <code>/<conversation_id>/message</code>	Sends a message in a conversation and updates the assistant's response
GET <code>/api/conversation/<conversation_id></code>	Fetches the complete chat history for a specific conversation
GET <code>/api/conversations</code>	Lists all conversations for the user, sorted by recent updates
DELETE <code>/conversation/<conversation_id></code>	Deletes a conversation by ID

Table 4.6: Core API Endpoints

- Each API endpoint implements detailed validation, structured error responses, and thorough logging.

4. Ingestion and Retrieval from the API

- **Document Upload (/api/upload):**
 - Supports text and binary file uploads (max 10MB).
 - Stores files as binary in MongoDB (`documents` collection).
 - Triggers the FAISS index rebuilding process in the background upon successful upload.
- **Question Answering (/api/ask):**
 - Retrieves the user's FAISS index.
 - Performs retrieval-augmented response generation using the Mistral AI LLM.
 - Incorporates query rewriting for better retrieval quality.

5. Conversation Management

- A sophisticated conversation state system is built on top of MongoDB using the `conversation_states` collection:
 - **State Saving:**
 - * Each message and context is stored per thread (user+conversation) using `save_state(thread_id, state)`.
 - * Conversation states persist over sessions and API requests.
 - **State Loading:**
 - * On each message, the system loads previous conversation history to maintain continuity.
 - **Conversation Graph:**
 - * The `LangGraph StateGraph` manages the conversation flow:

- `retrieve_context` Node: Retrieves documents relevant to the current message.
 - `generate_response` Node: Generates an AI answer and evaluates quality (hallucination and relevance).
 - `End` Node: Marks conversation turn completion.
- * This architecture allows dynamic and intelligent conversation management that evolves with the user's questions.

6. Web Search Fallback

- When document retrieval fails (no relevant documents found), EngageMind gracefully degrades to:
 - Perform a Tavily web search for the rewritten query.
 - Retrieve top snippets.
 - Use those snippets as temporary context to generate an answer.
 - Mark the answer with appropriate disclaimers to inform the user.
- This fallback ensures that the system remains helpful even when personalized documents are not available.

7. Error Handling and Robustness

- Robust error handling mechanisms are incorporated throughout:
 - **Custom Rate Limit Handler:** Returns a structured 429 response on exceeding limits.
 - **Fallback Recovery:** Automatic web fallback in case of document retrieval failure.
 - **Comprehensive Logging:** All operations (uploads, chats, retrievals) are logged with timestamps and user IDs for traceability.
 - **Graceful MongoDB Connection Handling:** Database interactions are protected with exception handling to avoid server crashes.

Key Functions and Their Roles

Function	Purpose
<code>create_app()</code>	Initializes and configures the Flask app
<code>register_routes(app, mongo)</code>	Defines and registers all API routes
<code>save_state(thread_id, state)</code>	Saves conversation state into MongoDB
<code>load_state(thread_id)</code>	Loads conversation state from MongoDB
<code>build_conversation_graph()</code>	Defines the LangGraph flow for retrieval and response generation
<code>perform_web_fallback(state, user_id, query)</code>	Performs web search fallback in absence of document context

Table 4.7: Key Functions in Server and API Setup

4.5.5 FAISS Index Management

Purpose

To enable personalized retrieval for each user, EngageMind dynamically manages multiple FAISS indices. Each user's uploaded documents are embedded, indexed, and isolated within a dedicated FAISS vector store. This design supports efficient retrieval, scalability, and data separation across users.

Index Management Workflow

The system's FAISS management includes the following operations:

1. Loading Existing Indices

- When a user submits a query, the server first checks if a FAISS index already exists for that user under a user-specific directory inside `faiss_index_dir/`.
- The index is loaded from disk using FAISS's serialization capabilities.
- If the index does not exist, the system prompts the user to upload documents first.

2. Creating New Indices

- When a user uploads documents (through `/api/upload`), a new FAISS index is created if none exists.
- Uploaded files are parsed, chunked, embedded, and stored as a new vector index linked to the user's ID.

3. Saving Indices

- FAISS indices are saved persistently to disk after ingestion or updates.
- This persistence ensures that user knowledge bases survive server restarts without needing re-ingestion.
- **Saved Files:**
 - `index.faiss` — FAISS vector data (binary format)
 - `index.pkl` — Metadata for each document chunk (stored alongside vectors)

4. Organizing User-specific Directories

- Each user's index is stored inside a separate directory:

```
faiss_index_dir/  
  {user_id}/  
    index.faiss  
    index.pkl
```

- This organization maintains strong user isolation, simplifies backup strategies, and improves index loading speed by scoping retrieval to relevant user directories only.

Technical Insights

- **Dynamic Index Loading:**
 - Only the requested user's index is loaded into memory on-demand, reducing RAM consumption.

- **Consistency with Ingestion:**

- Every ingestion operation updates the user's local FAISS directory immediately after successful embedding generation.

- **Support for Incremental Growth:**

- Though not active yet, the architecture can be extended to append new documents to an existing index without a full rebuild.

4.5.6 Evaluation Module

Purpose

The Evaluation Module is designed to assess the quality of responses generated by EngageMind's RAG system. It ensures that:

- Retrieved context documents are relevant to the query.
- Generated answers are grounded in the retrieved content and avoid hallucinations.

By applying automated grading techniques, the system maintains high-quality, trustworthy AI interactions.

Components and Functions

The Evaluation Module provides the following key capabilities:

1. Query Rewriting

- **Function:** `rewrite_query(question: str, llm)`
- **Purpose:**
 - Reformulates vague or unclear user queries into clearer, retrieval-friendly questions.
 - Enhances the effectiveness of similarity search and increases retrieval precision.
- **Workflow:**

- A rewriting prompt template (`REWRITE_PROMPT`) instructs the LLM to cleanly rewrite the original query.
- The rewritten query is then used for document retrieval.
- **Importance:**
 - This step significantly improves retrieval results, especially when users submit short, context-dependent, or ambiguous queries.

2. Hallucination Grading

- **Function:** `grade_hallucination(docs: List[Document], answer: str, llm)`
- **Purpose:**
 - Evaluates whether the AI's generated answer is truly grounded in the provided retrieved documents.
- **Workflow:**
 - The content from the retrieved documents is concatenated (up to a configurable character limit).
 - A grading prompt (`HALLUCINATION_PROMPT`) asks the LLM to judge if the answer is "grounded" or "hallucinated" based on this context.
 - The output is either "yes" (grounded) or "no" (hallucinated).
- **Importance:**
 - Helps detect when the model makes unsupported claims not backed by retrieved knowledge.

3. Answer Relevance Grading

- **Function:** `grade_answer_relevance(question: str, answer: str, llm)`
- **Purpose:**
 - Checks whether the AI's generated answer actually resolves the user's original question.

- **Workflow:**
 - The question and generated answer are passed into a relevance grading prompt (`ANSWER_RELEVANCE_PROMPT`).
 - The LLM determines if the answer meaningfully addresses the user's need with a "yes" or "no" result.
- **Importance:**
 - Ensures that answers are not just fluent, but relevant and useful to the user.

Key Technical Details

Component	Description
<code>ChatPromptTemplate</code>	Defines structured LLM prompts for rewriting and grading tasks
<code>StrOutputParser</code>	Parses and formats LLM outputs into clean strings
<code>max_context_len</code>	Limits the context size to avoid overwhelming the grading model

Table 4.8: Key Components in Evaluation Module

4.5.7 Summary of RAG Backend

The Retrieval-Augmented Generation (RAG) backend of EngageMind is a modular system designed for scalable, personalized conversational AI. It integrates document ingestion, semantic retrieval, secure APIs, and response evaluation to deliver accurate, user-specific interactions. Each component optimizes performance and data isolation, supporting multi-user scalability.

Key components include:

- **Ingestion Pipeline:** Converts uploaded documents into FAISS embeddings using `MistralAIEmbeddings`, supporting OCR for non-text formats.

- **Retrieval Pipeline:** Retrieves document chunks via FAISS, enhanced by Mistral LLM query rewriting for precise multi-turn conversations.
- **Server and API Setup:** Offers a Flask-based REST API for uploads and conversations, with JWT authentication and MongoDB state persistence.
- **FAISS Index Management:** Maintains user-specific FAISS indices, loaded on-demand for efficient retrieval.
- **Evaluation Module:** Ensures response quality by grading hallucination and relevance using Mistral LLM.

MongoDB stores documents, chat histories, and conversation states, while a LangGraph workflow manages dialogue flow. A Tavily web search fallback ensures functionality when documents are unavailable. With robust error handling and logging, the backend provides a reliable user experience, ready for future enhancements.

The next sections cover the User Authentication Backend and Frontend Application, completing the EngageMind system.

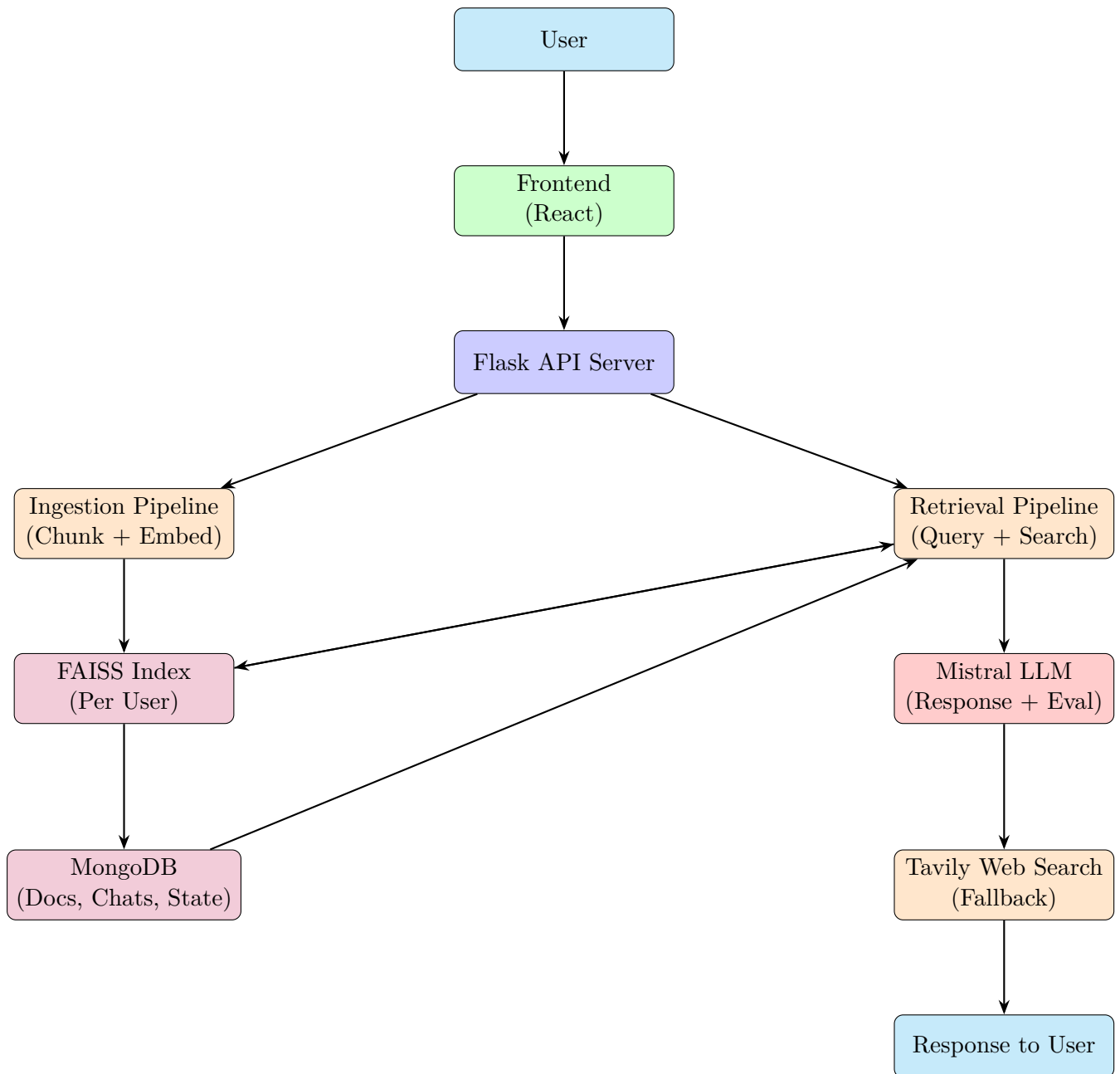


Figure 4.2: Flow Diagram of EngageMind RAG Backend Operations

4.6 UserAuth Backend Implementation

The User Authentication Backend of EngageMind is a robust, secure system built with Node.js and Express.js, designed to manage user authentication, authorization,

profile management, and password recovery. It supports both local and Google OAuth-based accounts, leveraging MongoDB with Mongoose for data persistence, Passport.js for authentication, and the Resend API for email-based workflows. The backend ensures seamless user interactions with strong security and role-based access control.

Key functionalities include:

- **Authentication:** Supports local email/password and Google OAuth via Passport.js, with JWT-based session management.
- **Email Verification:** Requires email confirmation for new accounts, sent via Resend API.
- **Password Recovery:** Enables secure password resets through time-bound email links.
- **Profile Management:** Allows users to update bio, avatar, and social links.
- **Access Control:** Enforces role-based permissions (User/Admin) for restricted endpoints.

Project Structure

The backend is organized as follows:

- `config/`: Database and Passport.js configurations.
- `middleware/`: Authentication and role-based access middleware.
- `models/`: Mongoose schema for user data.
- `routes/`: API endpoint definitions.
- `server.js`: Main server file.
- `.env`: Environment variables (e.g., `MONGO_URI`, `GOOGLE_CLIENT_ID`, `RESEND_API`).

Configuration

- **Database:** Connects to MongoDB via Mongoose, using `MONGO_URI` from environment variables (`config/db.js`).

- **Authentication:** Configures Passport.js with JWT and Google OAuth strategies (`config/passport.js`). JWT validates protected routes, while Google OAuth creates or logs in users based on `googleId`.

User Model

The `User` schema (`models/userModel.js`) defines: Timestamps (`createdAt`,

Field	Type	Description
<code>username</code>	String	Unique username, required
<code>email</code>	String	Unique email, required
<code>password</code>	String	Required for local accounts
<code>provider</code>	String (enum)	<code>local</code> or <code>google</code>
<code>googleId</code>	String	Google OAuth ID (optional)
<code>role</code>	String (enum)	<code>user</code> (default) or <code>admin</code>
<code>verified</code>	Boolean	Email verification status
<code>bio</code>	String	User biography
<code>avatar</code>	String	Profile picture URL
<code>socialLinks</code>	Object	Social media links (Twitter, LinkedIn, GitHub)
<code>verificationToken</code>	String	Email verification token
<code>passwordResetToken</code>	String	Password reset token

Table 4.9: User Model Schema

`updatedAt`) are automatically added.

Middleware

- **Authentication** (`middleware/authMiddleware.js`): Verifies JWT tokens, attaching the user to the request.
- **Role-Based** (`middleware/roleMiddleware.js`): Restricts access to admin-only endpoints, returning 403 if unauthorized.

Endpoint	Method	Description	Auth Required
/api/register	POST	Register user, send verification email	No
/api/login	POST	Authenticate user, issue JWT	No
/api/auth/google	GET	Redirect to Google OAuth	No
/api/auth/google/callback	GET	Handle Google OAuth callback	No
/api/logout	POST	Logout user	Yes
/api/forgot-password	POST	Send password reset link	No
/api/reset-password/:token	POST	Reset password	No
/api/verify-email/:token	GET	Verify email	No
/api/admin/users	GET	Fetch all users (admin only)	Yes (admin)

Table 4.10: Authentication Backend API Endpoints

API Endpoints

Application Flow

- **Registration:** Users submit credentials, receive a verification email via Resend API, and must verify before activation.
- **Google Login:** Users authenticate via Google OAuth, with accounts created or logged in based on `googleId`.
- **Password Reset:** Users request a reset link, sent via Resend API, and submit a new password using a time-bound token.
- **Protected Actions:** JWT tokens secure access, with admin endpoints restricted to `role: admin`.

Error Handling

- **Structured Responses:** APIs return appropriate HTTP codes (e.g., 401 for invalid JWT, 403 for forbidden access).

- **Validation:** Failures (e.g., duplicate emails) yield 400 Bad Request responses with clear messages.
- **Logging:** Errors are logged for debugging and monitoring.

4.6.1 User Authentication Backend Summary

The User Authentication Backend of EngageMind is a secure, scalable system built with Node.js and Express.js, enabling robust user management for local and Google OAuth-based accounts. It leverages MongoDB with Mongoose for data persistence, Passport.js for authentication, and the Resend API for email workflows, ensuring seamless registration, login, and profile management with strong security and role-based access control.

Key components include:

- **Authentication:** Supports local email/password and Google OAuth via Passport.js, with JWT tokens securing sessions.
- **Email Verification:** Enforces email confirmation for new accounts using Resend API.
- **Password Recovery:** Provides time-bound password reset links sent via email.
- **Profile Management:** Allows updates to bio, avatar, and social links, stored in MongoDB.
- **Access Control:** Restricts admin endpoints to users with `role: admin`.

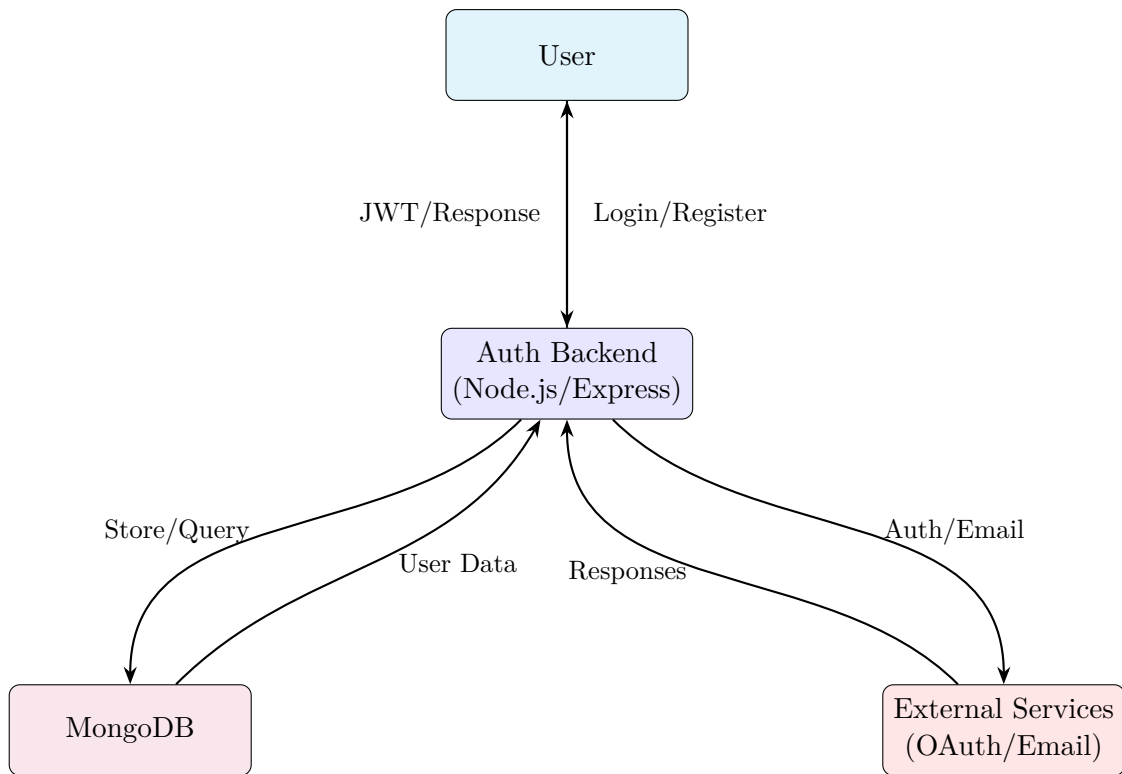


Figure 4.3: Simplified Flow Diagram of EngageMind Authentication Backend

4.7 Frontend Implementation

4.7.1 Project Overview

This project is a React.js-based frontend application designed to facilitate user authentication, profile management, secure chat functionalities, and file uploads to enhance chat interactions. It employs protected routing to ensure secure access to sensitive areas and incorporates robust handling of error and loading states. The application is modularized into components, pages, services, and API modules, promoting scalability and maintainability.

4.7.2 Application Bootstrapping: `index.js`

The application initializes in `index.js`, where the root component `<App />` is rendered into the DOM element identified by `root`. Styles are applied through `index.css`, which integrates TailwindCSS and custom styles for consistent presentation.


```

1 import './index.css';
2 import App from './App';
3
4 const root = ReactDOM.createRoot(document.getElementById('root'));
5 root.render(
6   <React.StrictMode>
7     <App />
8   </React.StrictMode>
9 );

```

4.7.3 Routing Architecture: App.js

Navigation is managed using React Router v6, with all routes defined in `App.js`. The routing architecture distinguishes between public and protected routes, ensuring that sensitive areas are accessible only to authenticated users. Below is the detailed routing structure:

Type	Path	Component	Description
Public	/	HomePage	Landing page for all users
Public	/login	LoginPage	User login interface
Public	/register	RegisterPage	User registration form
Public	/reset-password	ResetPasswordPage	Password reset interface
Public	/forgot-password	ForgotPasswordPage	Forgot password request page
Public	/verify-email	VerifyEmailPage	Email verification confirmation
Public	/auth-success	AuthSuccess	Email verification success page
Public	/access-denied	AccessDeniedCard	Restricted access notification
Protected	/chat	ChatPage	Secure chat interface
Protected	/profile	UserProfile	User profile viewing page
Protected	/edit-profile	EditProfile	Profile editing interface
Fallback	*	Redirect to /	Redirects undefined routes to home

Table 4.11: Routing Architecture

4.7.4 User Authentication Flow

The authentication system is token-based, relying on `localStorage` to persist the authentication token across sessions. This token is used to authenticate API requests and

control access to protected routes.

Authentication Token Behavior

- **Key:** token
- **Storage:** Stored in the browser's `localStorage` upon successful login or registration.
- **Usage:**
 - Attached to every authenticated API request via Axios interceptors in the `Authorization` header.
 - Checked by protected routes to determine access eligibility.

AxiosAuth Configuration

The Axios instance for authentication, defined in `/src/api/axiosAuth.js`, is configured to include the authentication token in every request header. The base URL for authentication-related APIs is `http://localhost:5000`.

```
1 const token = localStorage.getItem('token') || 'fallback-token';  
2 config.headers.Authorization = `Bearer ${token}`;
```

- **Fallback Mechanism:** If no token is found in `localStorage`, a fallback token is used, primarily for development purposes.
- **Interceptor Behavior:** The interceptor modifies the request configuration before it is sent, ensuring the `Authorization` header is always present for authenticated endpoints.

Authentication Pages and Step-by-Step Flow

- **LoginPage:**
 1. User enters credentials (e.g., email and password).
 2. Form submission triggers an API call: `axiosAuth.post('/login', { email, password })`.

3. On success, the backend returns a JSON response containing the `token`.
4. Token is saved: `localStorage.setItem('token', response.data.token)`.
5. User is redirected to `/profile` or `/chat` based on application logic.
6. **Error Handling:** If credentials are invalid, an error message is displayed via the `ErrorDisplay` component.

- **RegisterPage:**

1. User submits registration data (e.g., username, email, password).
2. API call: `axiosAuth.post('/register', { username, email, password })`.
3. Backend initiates email verification by sending a confirmation link.
4. If immediate token issuance is configured, it is stored: `localStorage.setItem('token', response.data.token)`.
5. User is redirected to `/verify-email` or prompted to check their email.
6. **Error Handling:** Duplicate email or weak password triggers an error displayed to the user.

- **ForgotPasswordPage:**

1. User enters their email address.
2. API call: `axiosAuth.post('/forgot-password', { email })`.
3. Backend sends a password reset link to the provided email.
4. Success message is shown; no token is involved at this stage.
5. **Error Handling:** Invalid email results in an error notification.

- **ResetPasswordPage:**

1. Accessed via a tokenized URL (e.g., `/reset-password?token=xyz`).
2. User enters a new password.
3. API call: `axiosAuth.post('/reset-password', { token, newPassword })`.
4. On success, user is redirected to `/login`.

5. **Error Handling:** Expired or invalid token displays an error.

- **VerifyEmailPage:**

1. Accessed via a verification URL (e.g., `/verify-email?token=abc`).

2. API call: `axiosAuth.get('/verify-email?token=abc')`.

3. Backend confirms the email, potentially issuing a final `token`.

4. Redirects to `/auth-success`.

5. **Error Handling:** Invalid token shows an error message.

4.7.5 ProtectedRoute Logic

The `ProtectedRoute` component enforces access control for protected routes by verifying the authentication token's presence.

```
1 const isAuthenticated = !!localStorage.getItem('token');
```

Detailed Behavior

- **Token Check:**

- The condition `!!localStorage.getItem('token')` evaluates to `true` if a token exists, `false` otherwise.

- **Access Scenarios:**

- **Authenticated User:** If `isAuthenticated` is `true`, the requested component (e.g., `ChatPage`) is rendered.

- **Unauthenticated, Accessing `/chat`:** If `isAuthenticated` is `false`, the `AccessDeniedCard` is displayed with a registration link (`/register`).

- **Unauthenticated, Other Protected Routes:** Redirects to `/login` using React Router's navigation.

- **Loading State:** During initial data fetching (e.g., conversations for `/chat`), a `LoadingSpinner` is shown.

- **Error Handling:** If data fetching fails, the `ErrorDisplay` component renders with the error details.

4.7.6 Chat System Detailed Flow

The chat system enables users to create conversations, send messages, and upload documents, all authenticated via the same `token` used elsewhere. It relies on the `axiosChat` API instance.

LocalStorage Behavior

- **Key:** `token` (shared with authentication).
- **Usage:** Axios interceptors attach `Authorization: Bearer <token>` to all chat-related requests.

`axiosChat` Configuration

Defined in `/src/api/axiosChat.js` with a base URL of `http://localhost:5001`.

```
1 const token = localStorage.getItem('token');  
2 config.headers.Authorization = `Bearer ${token}`;
```

- **Interceptor:** Ensures every request includes the token, failing which the request is rejected by the backend with a 401 status.

Chat Functionalities

Fetch Conversations

- **Function:** `fetchConversations()`
- **API Call:** `axiosChat.get('/api/conversations')`
- **Step-by-Step:**
 1. Triggered on `ChatPage` load if authenticated.

2. Request sent with `Authorization` header.
 3. Backend returns an array of conversation summaries (e.g., `{id, title, lastMessage}`).
 4. Data populates `Sidebar` and `ChatList` components.
- **Error Handling:** Network failure or 401 status displays `ErrorDisplay`.

Create New Conversation

- **Function:** `createConversation()`
- **API Call:** `axiosChat.post('/api/conversation')`
- **Step-by-Step:**
 1. User clicks a "New Chat" button in `Sidebar`.
 2. POST request sent with `Authorization` header.
 3. Backend responds with a new `conversation_id`.
 4. Frontend updates `ChatList` with the new conversation.
- **Error Handling:** Failure (e.g., server error) shows an error message.

Fetch Conversation Messages

- **Function:** `fetchConversationMessages(conversationId)`
- **API Call:** `axiosChat.get('/api/conversation/${conversationId}')`
- **Step-by-Step:**
 1. User selects a conversation from `ChatList`.
 2. GET request includes the specific `conversationId`.
 3. Backend returns message history (e.g., `{id, sender, content, timestamp}`).
 4. Messages render in `ChatWindow`.
- **Error Handling:** Invalid `conversationId` or 404 triggers `ErrorDisplay`.

Send a Message

- **Function:** `sendMessage(conversationId, message)`
- **API Call:** `axiosChat.post('/api/conversation/${conversationId}/message', { message })`
- **Step-by-Step:**
 1. User types a message in `ChatWindow` and submits.
 2. POST request sends the message payload.
 3. Backend processes and returns a bot response.
 4. Both user message and bot reply update `ChatWindow`.
- **Error Handling:** Failed send displays an error; message remains unsent.

Ask a Question (Outside Conversation)

- **Function:** `askQuestion(question)`
- **API Call:** `axiosChat.post('/api/ask', { question })`
- **Step-by-Step:**
 1. User submits a question via a global input (e.g., in `Navbar`).
 2. POST request sent without a `conversationId`.
 3. Backend provides an immediate response.
 4. Response displayed in a temporary UI element.
- **Error Handling:** Server error shows a notification.

Upload a Document

- **Function:** `uploadDocument(file)`
- **API Call:** `axiosChat.post('/api/upload', formData, { headers: {'Content-Type': 'multipart/form-data'}})`

- **Step-by-Step:**

1. User selects a file in `DocumentUploader`.
2. File is packaged into a `FormData` object.
3. POST request sent with custom headers and `Authorization`.
4. Backend stores the file and returns a confirmation.
5. UI updates to reflect upload success.

- **Error Handling:** File size limits or type errors trigger `ErrorDisplay`.

4.7.7 Components Connection

The application comprises interconnected components, each with a specific role:

Component	Purpose
Navbar	Header navigation bar with links and user options
Footer	Static page footer with additional information
Layout	Structural wrapper for public-facing pages
LoadingSpinner	Visual indicator for ongoing operations
ErrorDisplay	Displays error messages with details
AccessDeniedCard	Warns of unauthorized access with registration link
ChatPage	Main interface for chat functionalities
ChatContainer	Combines sidebar and chat window layout
Sidebar	Displays list of user conversations
ChatList	Renders conversation summaries
ChatWindow	Real-time chat message display and input
DocumentUploader	Interface for file uploads in chats
LoginPage, RegisterPage	Authentication entry points
UserProfile, EditProfile	Profile viewing and editing pages

Table 4.12: Main Components and Their Purposes

4.7.8 Loading and Error Handling

Loading and error states are managed consistently across the application:

- **Initial Load:** If authenticated, `App.js` calls `fetchConversations()` to preload chat data.
- **Loading State:** `LoadingSpinner` appears during API calls.

- **Error State:**

```
1 if (error) return <ErrorDisplay error={error} />;
```

- Errors from API calls (e.g., 401, 404) are caught and rendered with custom messages.

4.7.9 LocalStorage Key List

The application utilizes the following `localStorage` keys:

Key	Purpose
token	Primary authentication token for user sessions
API_TOKEN (optional)	Testing token used exclusively in <code>axios.js</code>

Table 4.13: LocalStorage Keys

4.7.10 Data Flow Diagram

The application's data flow is outlined below in a textual representation:

User → (Login/Register) → token saved → Routes open
→ Access Chat → `fetchConversations` → Sidebar → Select conversation
→ `fetchConversationMessages` → ChatWindow → `sendMessage` → Chat updated
→ (Optionally) `uploadDocument` → enrich chat context
→ Profile → View/Edit profile

4.7.11 Chat Application Flow

This diagram illustrates the workflow of a chat application, starting with user authentication where the user logs in or registers, and a token is saved to enable access to various routes. Once routes are open, the user can access the chat functionality, which involves fetching conversations to display in the sidebar, selecting a conversation, and retrieving its messages to show in the chat window. Users can then send messages to update the chat or upload documents to enrich the chat context. Additionally, the user can navigate to their profile to view or edit their details, providing a complete overview of the application's core interactions

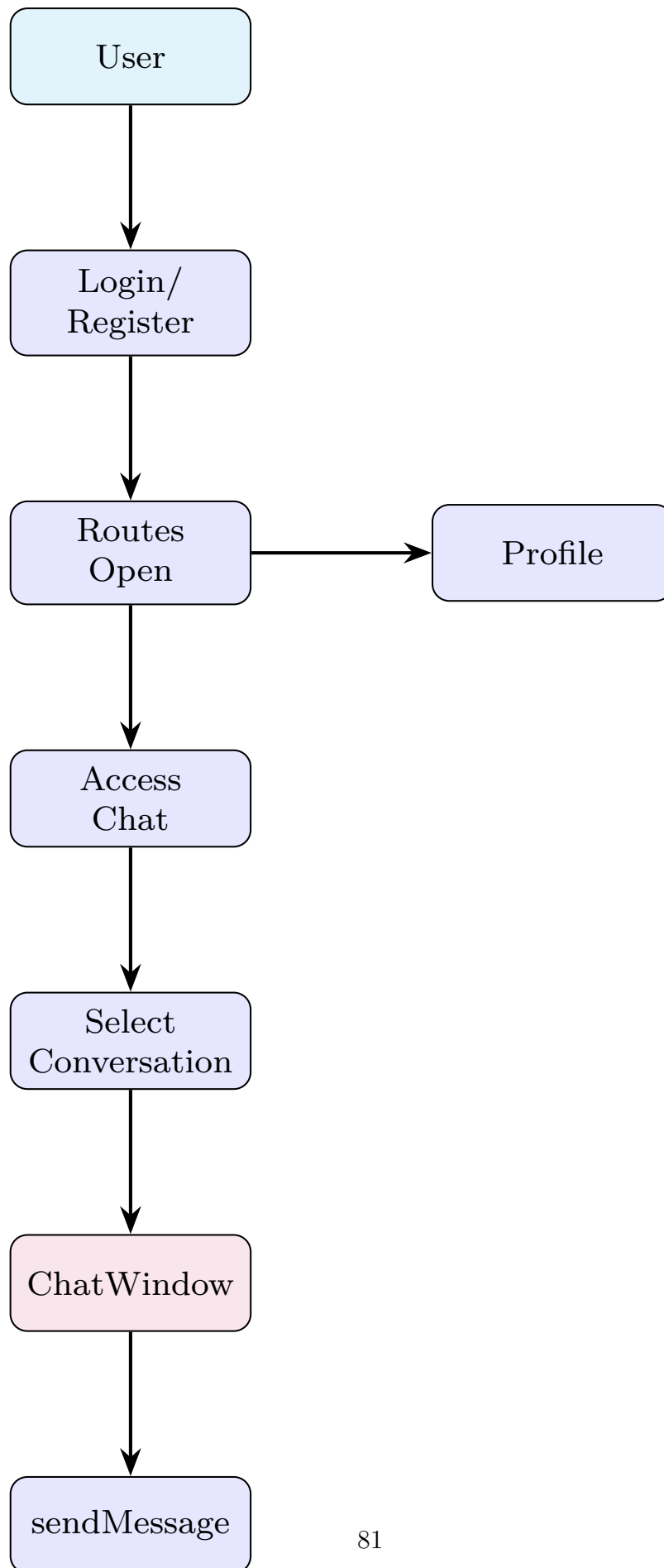


Figure 4.4: Chat System Flow

Acknowledgements

Bibliography

- [1] Alec Radford et al. “Language models are unsupervised multitask learners”. In: *OpenAI Blog* 1.8 (2019).
- [2] *GitHub: Where the world builds software*. <https://github.com/>. Accessed: 2025-04-25. 2024.
- [3] *Git - Distributed Version Control System*. <https://git-scm.com/>. Accessed: 2025-04-25. 2024.
- [4] *Homebrew - The Missing Package Manager for macOS (or Linux)*. <https://brew.sh/>. Accessed: 2025-04-25. 2024.
- [5] *MongoDB - The Developer Data Platform*. <https://www.mongodb.com/>. Accessed: 2025-04-25. 2024.
- [6] *Flask - Python Web Framework*. <https://flask.palletsprojects.com/>. Accessed: 2025-04-25. 2024.
- [7] *LangChain - Building Applications with LLMs through Composability*. <https://www.langchain.com/>. Accessed: 2025-04-25. 2024.
- [8] *LangGraph - State Machines for LLM Applications*. <https://langchain-ai.github.io/langgraph/>. Accessed: 2025-04-25. 2024.
- [9] *FAISS - Facebook AI Similarity Search*. <https://faiss.ai/>. Accessed: 2025-04-25. 2024.
- [10] *PyTorch - An Open Source Machine Learning Framework*. <https://pytorch.org/>. Accessed: 2025-04-25. 2024.
- [11] *Express - Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com/>. Accessed: 2025-04-25. 2024.

- [12] *Node.js is an open source server environment.* <http://nodejs.org/>. Accessed: 2025-04-25. 2024.
- [13] *Passport.js - Simple, unobtrusive authentication for Node.js.* <http://www.passportjs.org/>. Accessed: 2025-04-25. 2024.
- [14] *JWT.io - JSON Web Tokens.* <https://jwt.io/>. Accessed: 2025-04-25. 2024.
- [15] *Resend - Email API for developers.* <https://resend.com/>. Accessed: 2025-04-25. 2024.
- [16] *Google OAuth 2.0 Authorization Framework.* <https://developers.google.com/identity/protocols/oauth2>. Accessed: 2025-04-25. 2024.
- [17] *React - A JavaScript library for building user interfaces.* <https://react.dev/>. Accessed: 2025-04-25. 2024.
- [18] *Vite - Next Generation Frontend Tooling.* <https://vitejs.dev/>. Accessed: 2025-04-25. 2024.
- [19] *Tailwind CSS - Rapidly build modern websites without ever leaving your HTML.* <https://tailwindcss.com/>. Accessed: 2025-04-25. 2024.
- [20] *Redis - In-Memory Data Structure Store.* <https://redis.io/>. Accessed: 2025-04-25. 2024.
- [21] *Celery - Distributed Task Queue.* <https://docs.celeryq.dev/en/stable/>. Accessed: 2025-04-25. 2024.
- [22] *Mistral AI - Open-Weight Language Models.* <https://mistral.ai/>. Accessed: 2025-04-25. 2024.
- [23] *Tavily Search API - Real-time Web Data for LLMs.* <https://docs.tavily.com/>. Accessed: 2025-04-25. 2024.
- [24] *Google Cloud Console - Manage Cloud Resources.* <https://console.cloud.google.com/>. Accessed: 2025-04-25. 2024.

List of Figures

2.1	The landing page of the EngageMind RAG application, displaying the initial interface with prominent “Register” and “Login” buttons to guide user interaction.	17
2.2	Register Page	18
2.3	The login page of the EngageMind RAG application, featuring email/password login, Google sign-in, and password recovery options.	20
2.4	The EngageMind chat interface with conversation history, input field, and upload icon.	22
2.5	Example query entered in the chat interface.	23
2.6	System response based on the uploaded document.	24
2.7	System response using Tavily web search when no document is uploaded.	24
4.1	EngageMind System Architecture	42
4.2	Flow Diagram of EngageMind RAG Backend Operations	66
4.3	Simplified Flow Diagram of EngageMind Authentication Backend	71
4.4	Chat System Flow	81

List of Tables

4.1	MongoDB Collections	45
4.2	Flask API Endpoints	46
4.3	Express API Endpoints	46
4.4	Key Functions in Ingestion Pipeline	51
4.5	Key Components in Retrieval Pipeline	55
4.6	Core API Endpoints	57
4.7	Key Functions in Server and API Setup	60
4.8	Key Components in Evaluation Module	64
4.9	User Model Schema	68
4.10	Authentication Backend API Endpoints	69
4.11	Routing Architecture	72
4.12	Main Components and Their Purposes	79
4.13	LocalStorage Keys	80

List of Algorithms

List of Codes